```
# ============================================================================
# PACKAGE: chiyu-kernel  —— L0 契约 + L1 基础设施
# ============================================================================
# ----- FILE: packages/chiyu-kernel/pyproject.toml -----
[project]
name = "chiyu-kernel"
version = "0.1.0"
description = "ChiYu AI Native Foundation - L0 契约与 L1 基础设施内核"
requires-python = ">=3.11,<3.14"
dependencies = [
    "psycopg2-binary>=2.9",
    "pgvector>=0.2",
    "numpy>=1.26",
    "PyYAML>=6.0",
]

[project.optional-dependencies]
cpu = ["onnxruntime>=1.17", "tokenizers>=0.15"]
dev = ["pytest>=8.0", "pytest-cov>=5.0"]

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/__init__.py -----
"""chiyu-kernel —— ChiYu AI Native Foundation 的 L0 契约 + L1 基础设施内核。

授权分级
--------

    kernel    基础授权（本包）—— L0 契约 + L1 基础设施
    decision  核心授权（`chiyu-decision`，阶段 P2）—— KMD 裁决时序 / 九尺子 / 红线
    portal    可选装（`chiyu-portal`，阶段 P2）—— NL→PageSpec→渲染器

⛔ 消费方使用纪律
-----------------

只许 import `chiyu_kernel.contract` 与 `chiyu_kernel.api`。import 内部实现路径
（如 `chiyu_kernel.store.pg_store`）即违约——守 `tests/test_app_imports_contract_only.py`。
理由：内部实现要能在不破坏消费方的前提下重构（对外轨的 Cython 编译会改模块布局）。

⛔ 本包零领域字面量
-------------------

粒子类型取值、知识谓词、红线事项目录全部由消费方注入 `TenantProfile`，内核只声明形态。
守卫：`tests/test_contract_no_domain_literals.py`。
```

```
"""
from __future__ import annotations

__version__ = "0.1.0"
AUTHORIZATION_LEVEL = "kernel"
LICENSE_SCOPE = ("kernel",)

__all__ = ["__version__", "AUTHORIZATION_LEVEL", "LICENSE_SCOPE"]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/actions/__init__.py -----
"""行动面注册表（R1－R6 基数的可执行形式）—— ★ 新建，demo 未收敛。

⛔ 内容为**机制**（判定规则），非具体 Action 清单（那是 L4 消费方域内容）。
"""
from .registry import (ActionRegistry, ActionSpec, ACTION_MAX, ACTION_MIN)

__all__ = ["ActionSpec", "ActionRegistry", "ACTION_MIN", "ACTION_MAX"]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/actions/registry.py -----
"""Action 表面注册表（R1－R6 基数规则的**可执行形式**）—— ★ 新建，demo 未收敛。

⛔ 为什么要有注册表而不是继续加端点
──────────────────────────────────────
demo 的 59 个端点里出现"同一能力多端点并存"（如 `/api/retro/*` 4 个），
这是 R5（一能力一 Action）被违反的典型。加端点不会报错，所以没人拦得住 ——
除非有一个注册表在**注册期**就拒绝重复。

R1－R6（判定规则）：
    R1 一个业务域 5－9 个 Action（过多=CRUD 爆炸，过少=能力不足）
    R2 Action 名 = 动词 + 对象（`submit_request` 而非 `requestSubmit`）
    R3 读操作不单独占位（读并入对象的 query Action）
    R4 写操作必须有决策锚（decision_id）
    R5 一个能力一个 Action（同能力的多入口必须折叠）
    R6 高危写必须有 force 确认位

⛔ 零领域词：注册表只承载**判定机制**；具体 Action 清单是 L4（消费方域内容）。
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple

ACTION_MIN, ACTION_MAX = 5, 9
_NAME_RE = re.compile(r"^[a-z][a-z0-9]*(_[a-z0-9]+)+$")
_HIGH_RISK_HINTS = ("delete", "purge", "reset", "drop", "approve", "release")


@dataclass(frozen=True)
class ActionSpec:
    name: str
    capability: str                # R5：同一 capability 只能有一个 Action
```

```
    mutating: bool = False          # R4：写操作必须有决策锚
    high_risk: bool = False         # R6：高危写必须有 force
    has_force: bool = False

    def violations(self) -> List[str]:
        out: List[str] = []
        if not _NAME_RE.match(self.name):
            out.append(f"R2 命名不合规（须 verb_object）：{self.name}")
        if self.high_risk and not self.has_force:
            out.append(f"R6 高危写缺 force 确认位：{self.name}")
        if not self.high_risk and self.has_force:
            out.append(f"R6 非高危动作不应带 force 位：{self.name}")
        return out


class ActionRegistry:
    def __init__(self) -> None:
        self._by_name: Dict[str, ActionSpec] = {}
        self._by_capability: Dict[str, str] = {}

    def register(self, spec: ActionSpec) -> None:
        if spec.name in self._by_name:
            raise ValueError(f"重复注册 Action：{spec.name}")
        owner = self._by_capability.get(spec.capability)
        if owner is not None:
            raise ValueError(
                f"R5 违反：capability {spec.capability!r} 已由 {owner!r} 占用，"
                f"新增 {spec.name!r} ⇒ 应折叠到既有 Action，而不是新开端点")
        self._by_name[spec.name] = spec
        self._by_capability[spec.capability] = spec.name

    def audit(self) -> Tuple[List[str], List[str]]:
        """返回 (errors, warnings)。errors 非空 ⇒ 表面不合规，不得发布。"""
        errors: List[str] = []
        warnings: List[str] = []
        n = len(self._by_name)
        if n < ACTION_MIN:
            warnings.append(f"R1 Action 数偏少（{n} < {ACTION_MIN}）")
        if n > ACTION_MAX:
            errors.append(f"R1 Action 数超限（{n} > {ACTION_MAX}）⇒ CRUD 爆炸")
        for spec in self._by_name.values():
            errors.extend(spec.violations())
        return errors, warnings

    def by_name(self, name: str) -> Optional[ActionSpec]:
        return self._by_name.get(name)

    def names(self) -> Tuple[str, ...]:
        return tuple(sorted(self._by_name))
```

```
__all__ = ["ActionSpec", "ActionRegistry", "ACTION_MIN", "ACTION_MAX"]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/api.py -----
"""chiyu-kernel 的**唯一公开门面**。

⛔ 消费方只许 import 下列 **sanctioned 面**（成文契约，见 `docs/sanctioned-import-surface.md`，
   由 `tools/check_import_surface.py` 机械门禁，⛔ 该门禁**对 foundation 自身同样生效**）：
     • `chiyu_kernel.api`（本模块，基础内核面）
     • `chiyu_kernel.contract`（L0 契约面，含其子模块如 `chiyu_kernel.contract.decision`）
     • `chiyu_kernel.approval`（决策审批闸面）
     • `chiyu_decision.api`（决策脊柱面：**`run_spine` 在此**，⛔ 不在本模块 ——
       `chiyu-decision` 是独立分发包，kernel 不得反向依赖它 ⇒ 决策能力请用
       `from chiyu_decision.api import run_spine`）
     • `chiyu_portal.api`（门户面，若授权）
   其余内部路径（如 `chiyu_kernel.store.pg_store`、`chiyu_kernel.vector.infer`）一律不属消费面。
   守 tests/test_app_imports_contract_only.py（T2 Step 8）与 tools/check_import_surface.py。
   为什么要门面而不是"直接 import 也行"：内部实现要能在不破坏消费方的前提下
   重构（T9 的 Cython 编译会改模块布局），没有门面就只能靠"谁都不敢改"。

⛔ **接线完整性是可核验的，不是承诺**（2026-09-21 T5 收口时补）
──────────────────────────────────────────────────────────────────────────────
本模块的 `__all__` 必须**覆盖每个内层子包的 `__all__` 全集**。为什么这条要写成守卫：
"模块已建好但门面没暴露" 是一种**零报错**的缺陷 —— 消费方要么违约 import 内部路径，
要么以为内核没有该能力。守 `tests/test_api_facade_covers_all_layers.py`（逐层比对，
新增符号忘记导出即红）。这正是本会话发现 audit/events/llm/orchestration 四模块
"已写好却未接线"的同一类问题，故把它固化成判据。

⛔ `experimental` 状态透出：memory 内核标 experimental（T5′），消费方可据此
   判断是否可把本包写进 G1/A7 验收判据。
"""
from __future__ import annotations

from typing import Optional

from chiyu_kernel.contract import TenantProfile

# ─── L1 store：PG + pgvector 单一事实源 ──────────────────────────────────────
# ⛔ store 的 sanctioned 面由 `store/__init__.py.__all__` 单一事实源给出，
#    门面整族 re-export（此前只导出零散 9 项 ⇒ 消费方被迫摸内部子模块）。
from chiyu_kernel.store import (  # noqa: F401  （门面 re-export）
    DB_TESTING_ENV,
    DSN_ENV,
    DSN_TEST_ENV,
    FORBIDDEN_HOSTS,
    FORBIDDEN_ROLES,
    IDENT_MAX,
    SCHEMA_ENV,
    TESTING_FLAG_ENV,
    VALID_FORMS,
    DBConfigError,
```

```
    PgVectorStore,
    assert_no_free_jsonb,
    assert_no_hard_delete,
    assert_safe_identifier,
    close_all,
    close_schema,
    connect,
    created_schemas,
    cursor,
    ddl_statements,
    driver_available,
    driver_error,
    drop_ephemeral_schemas,
    drop_test_schema,
    ensure_schema,
    ephemeral_schemas,
    get_dsn,
    get_schema,
    in_testing_context,
    make_cursor,
    mask_dsn,
    pad_to_dim,
    parse_dsn,
    pg_to_vec,
    probe_schema_isolation,
    raw_dsn,
    register_ephemeral,
    shared,
    BACKEND_APP_NAME_PREFIX,
    DEFAULT_LOCK_TIMEOUT_MS,
    own_backend_identity,
    terminate_backends_sql,
    terminate_own_backends,
    validate_dsn,
    vec_to_pg,
    vector_index_ddl,
)

# ——— L0 contract 决策常量（N5：进 sanctioned 面，免消费方摸内部路径）———————————
from chiyu_kernel.contract.decision import (
    AUTONOMOUS_RELEASE_DEFAULT,
    EIGHT_ELEMENTS,
    RUBRIC_KEYS,
    RUBRIC_SCALE_MAX,
    SPINE_ORDER,
    validate_spine_order,
)

# ——— 版本与能力出口（N1/R11 + R4 合流：experimental 边界可机检）———————————————
from chiyu_kernel.version import (
```

```
    API_VERSION,
    CAPABILITIES,
    CONTRACT_VERSION,
    DEPRECATIONS,
    VALID_STATUSES,
    api_version,
    capabilities,
    capability_status,
    contract_version,
    deprecations,
    require_capabilities,
)
# ⛔ N1 `PACKAGE_VERSION` 只作兜底常量导入；门面真正透出的是 `__version__`
#    （动态取已装分发包版本，未装时回退 PACKAGE_VERSION）。
from chiyu_kernel.version import __version__, PACKAGE_VERSION  # noqa: F401

# ─── L1 vector：离线向量（零 torch）──────────────────────────────────────────
from chiyu_kernel.vector import (
    DEFAULT_MIN_COVERAGE,
    HASH_DIM,
    HASH_PROVIDER,
    InferBackend,
    OnnxRunner,
    OnnxUnavailable,
    CoverageReport,
    artifact_info,
    available as vector_available,
    backend as vector_backend,
    backfill_plan,
    clear_cache,
    compute_coverage,
    config_dim,
    content_hash,
    cosine,
    get_runner,
    hash_signature,
    hash_vector,
    info as vector_info,
    is_available as vector_is_available,
    is_padded_tag,
    resolve as vector_resolve,
    resolve_model_dir,
    stable_stringify,
    tag_for_dim,
)

# ─── L0/L1 ontology：粒子运行时 + 写时三钩子 + 受控关系图 ──────────────────────
from chiyu_kernel.ontology import (
    BackfillChannel,
    OntologyGraph,
```

```
    Particle,
    ensure_embedding,
    ensure_tsvector,
    normalize,
    ontology_sync,
)

# ─── L0/L1 context：需求侧编码 + L1–L4 装配 + 三级降级召回 ──────────────────────
from chiyu_kernel.context import (
    CONTEXT_DIMENSIONS,
    CONTEXT_LAYERS,
    GAP_INSUFFICIENT_EXPERIENCE,
    GAP_LOW_COVERAGE,
    GAP_MISSING_DATA,
    GAP_MISSING_ENTITY,
    GAP_MISSING_GATE,
    GAP_MISSING_HITL,
    KNOWLEDGE_LAYERS,
    KNOWLEDGE_NEEDS_DEFAULT_DIMS,
    LAYER_LABELS_ZH,
    LAYER_NAMES,
    MAX_DATASOURCES,
    MAX_ENTITIES,
    MAX_QUERY_TERMS,
    MAX_REQUIRED_FIELDS,
    MIN_EMBEDDING_COVERAGE,
    MIN_PRECEDENTS,
    TIER_FTS,
    TIER_FTS_VECTOR,
    TIER_NONE,
    TIER_ORDER,
    TIER_VECTOR,
    VTIER_FTS,
    VTIER_FTS_VECTOR,
    VTIER_NONE,
    VTIER_ORDER,
    VTIER_VECTOR,
    AssemblyResult,
    KnowledgeNeed,
    KnowledgeNeeds,
    RecalledItem,
    RetrievalEvidence,
    RetrievalResult,
    SupplyDemandGap,
    SupplyReport,
    assemble,
    build_knowledge_needs,
    check_supply_demand,
    degraded_recall,
    missing_context_dimensions,
```

```
    to_supply_layer,
)

# ─── L0 actions：R1–R6 注册表（机制，非 Action 清单）──────────────────────────
from chiyu_kernel.actions import ACTION_MAX, ACTION_MIN, ActionRegistry, ActionSpec

# ─── L0/L1 orchestration：任务状态机 + 派发参数 + 智能体注册闸 ──────────────────
from chiyu_kernel.orchestration import (
    ALLOWED_TRANSITIONS,
    BLOCK_KINDS,
    DISPATCH_CFG_DEFAULTS,
    DISPATCH_CFG_SECTION,
    FAILURE_LIMIT,
    IDEM_PREFIX,
    MAX_REDO,
    MIN_SUCCESS_LEN,
    SYSTEM_ANCHOR_PREFIX,
    SYSTEM_TASK_KINDS,
    TASK_KINDS,
    TASK_STATES,
    TERMINAL_STATES,
    ZOMBIE_STALE_SECONDS,
    AgentRegistry,
    AgentRegistryError,
    AgentSpec,
    TaskStateError,
    assert_anchor,
    assert_block_kind,
    assert_reset,
    assert_result_ref,
    assert_state,
    assert_transition,
    derive_idem_key,
    dispatch_cfg_report,
    fold_history,
    is_stuck,
    is_terminal,
    nightly_idem_key,
    read_dispatch_cfg,
    summarize,
    system_anchor,
)

# ─── L1 orchestration（运行时）：Agent 接口 + AgentLoop（harness 移植）──────────
from chiyu_kernel.orchestration.agent_runtime import Agent, LiveAgentRegistry
from chiyu_kernel.orchestration.agent_loop import AgentLoop
from chiyu_kernel.orchestration.loop_guard import LoopGuard, StepVerdict
from chiyu_kernel.orchestration.subagent import SubagentDescriptor, SubagentPool
from chiyu_kernel.orchestration.team import (
    SharedTask, TeamMember, TeamRegistry, TeamSpec,
```

```
)

# ─── L1 evolution：平台自进化闸（§12 机制层，★ 内核新建）──────────────────────────
from chiyu_kernel.evolution import (
    BASELINE_FIELD,
    READONLY_PATH_PREFIXES,
    GuardViolation,
    KnobError,
    KnobSpec,
    KnobTable,
    Prescription,
    PrescriptionError,
    ReplayVerdict,
    SkillDraft,
    SkillDraftError,
    apply_atomic,
    assert_not_baseline_skill,
    assert_prescription_transition,
    assert_writable,
    begin_replay,
    classify_change,
    g2_suspend_on_redline,
    is_forbidden_change,
    meta_redline_hits,
    promote,
    roll_back,
    shadow_replay,
    validate_draft,
    ChannelNotReadyError,
    CheckResult,
    ReadinessReport,
    assert_channel_may_open,
    readiness_report,
)

# ─── L1 events：进程内事件总线（★ 内核新建）──────────────────────────────────────
from chiyu_kernel.events import (
    EventBus,
    PublishReport,
    Subscriber,
    counting_subscriber,
    log_subscriber,
)

# ─── L1 audit：SHA-256 链式审计（★ 内核新建）──────────────────────────────────────
from chiyu_kernel.audit import (
    EMPTY_PLACEHOLDER,
    VerifyResult,
    build_chain,
    canonical,
```

```
    chain_hash,
    verify_chain,
)

# ─── L1 llm：档位策略（网关属消费方域内容，见 REFERENCE §K 待批项）──────────────
from chiyu_kernel.llm import (
    DEFAULT_PARALLEL,
    DEFAULT_TIER,
    TIERS,
    is_cloud_endpoint,
    normalize_tier,
    resolve_storyline_policy,
    tier_spec,
)

# ─── L1 llm（运行时）：provider 抽象（harness 移植）──────────────────────────────
from chiyu_kernel.llm.client import (
    ToolCall, LlmResponse, LlmProvider, StubProvider, DeepSeekProvider, build_provider,
)

# ─── L1 harness 原生桥：内核能力暴露为 agent 工具（harness 移植）──────────────────
from chiyu_kernel.harness_native_tools import (
    register_native_tools, native_redline_guard,
)

# ─── L1 plugins：Phase A「Everything is a Plugin」能力注册表 ──────────────────────
from chiyu_kernel.plugins import (
    BUILTIN_PLUGINS,
    DEFAULT_MODE_ID,
    MODE_PRESETS,
    PluginRegistry,
    PluginSpec,
    WorkMode,
    active_plugins,
    disabled_plugins,
    mode_ids,
    register_builtins,
    resolve_work_mode,
)

# ─── L1 session：会话日志（harness 移植）──────────────────────────────────────
from chiyu_kernel.session import SessionEvent, SessionLog, SessionManager

# ─── L1 tools：模型面工具注册 + 守卫管线（harness 移植）──────────────────────────
from chiyu_kernel.tools import (ToolSpec, ToolRegistry, ToolNotFound, ScopeDenied, GuardDenied)
from chiyu_kernel.tools.fs import SandboxedFs, PathEscapeError, register_fs_tools
from chiyu_kernel.tools.shell import (
    CommandRejected, run_command, TerminalSession, register_shell_tools,
)
```

```
# ─── L2 skills：技能注册表（harness 移植 • dsh skill 能力吸收）──────────────────────
from chiyu_kernel.skills import (
    SkillSpec, SkillRegistry, SkillConflict, is_skill_name, load_catalog,
)

# ─── L2 plan/todo：计划与待办数据模型（harness 移植 • dsh plan/todo 能力吸收）──
from chiyu_kernel.plan import (
    TodoItem, TodoList, PlanStep, Plan, PlanMode, PlanStateError,
)

# ─── L2 mcp：MCP 工具桥（harness 移植 • dsh mcp-client 能力吸收）──────────────────────
from chiyu_kernel.mcp import (
    McpToolDef, McpTransport, InMemoryMcpServer, McpBridge, normalize_mcp_name,
)
# ─── L2 hooks：钩子总线（harness 移植 • dsh hook-protocol 能力吸收）──────────────────
from chiyu_kernel.hooks import HookBus, HookResult, matches
# ─── L2 interaction：审批闸门（harness 移植 • dsh interaction 能力吸收）──────────
from chiyu_kernel.interaction import ApprovalGate, ApprovalRequest
# ─── L2 workflow：声明式编排数据模型（harness 移植 • dsh workflow 能力吸收）──
from chiyu_kernel.workflow import (
    WORKFLOW_STEP_STATES, WorkflowRunId, WorkflowPhase, WorkflowMeta,
    WorkflowStep, WorkflowResult, WorkflowStopReason, WorkflowError,
    is_fatal_workflow_error, WorkflowStateError, Workflow, WorkflowRunner,
    register_workflow_tools, WorkerRun,
)

# ─── L2 scenario：可运行场景包（模型 / 存储 / MCP 工具面，双入口共用同一写闸）──
from chiyu_kernel.scenario import (
    LEVEL_IDS, KNOWLEDGE_LAYERS, ScenarioPack, ScenarioPackError,
    ScenarioStep, from_dict as scenario_from_dict,
    ScenarioPackStore, ScenarioPackLockDenied, build_in_memory_server
    as scenario_build_in_memory_server,
)
# ─── L0 domain：领域注入 SDK（WS6 关键石：TenantProfile + ActionRegistry + ScenarioPack 统一校验聚合）
from chiyu_kernel.domain import (
    DomainBundle, DomainInjectionError, inject_domain, profile_from_dict,
)
# ─── L2 persist：标准 persister + 记忆网 canonical writer（WS2：决策落库机制 + 7 谓词边）
from chiyu_kernel.persist import (
    EVENT_TYPE_DECISION, DecisionRecord, judge_worthiness,
    write_decision_memory_net, PgVectorStorePersister,
    # B 双轨记忆运行时（LOG/NOTE 双轨 + 蒸馏侧两闸）+ F 闭环回流白名单
    DistillReport, InMemoryMemoryLog, InMemoryNoteStore, MemoryLogEntry,
    MemoryLogStore, MemoryNote, NoteStore, PRESCRIBABLE_PREDICATES,
    distill, is_prescribable, note_hash, parse_ts,
)
# ─── L2 graph：图存储抽象（WS8：单源双视图只读派生，纯 PG CTE 参考实现）
from chiyu_kernel.graph import (
    DECISION_TIME_BASIS, MAX_TRAVERSAL_HOPS, TIMELINE_SOURCES,
    GraphEdge, TraversalResult, GraphStore,
```

```
    InMemoryGraphStore, PgCteGraphStore,
    relation_view, timeline_view, decision_backlinks,
)
# ─── L2 mcpserver：MCP 服务端骨架（WS7；注册守卫 + 零信任鉴权 + 两阶段确认） ──
from chiyu_kernel.mcpserver import (
    RISKS, McpServerError, ToolNotExposed, AuthDenied, ConfirmRequired,
    ConfirmTokenInvalid, ExposedTool, canonical_args, issue_confirm_token,
    verify_confirm_token, McpServer, StdioTransport, allow_principals,
)
# ─── L2 approval：决策脊柱审批闸（WS3；硬约束先于置信度 / A 红线优先） ──
#     ⛔ 与 L2 interaction.ApprovalGate 不同层（决策落库前裁定 vs 工具执行前人工确认），
#       命名加 Decision 前缀防门面静默覆盖（同 vector_* 前缀纪律）。
from chiyu_kernel.approval import (
    DecisionApprovalGate, DecisionApprovalVerdict, DecisionVerdictState,
    RedlinePriorityGate, DecisionPassThroughGate,
)
# ─── L2 canary：决策落库三判据统一判定式（N7；判据口径收归基座，消费方只注接线） ──
from chiyu_kernel.canary import CanaryReport, canary_probe  # noqa: F401
# ─── L2 scenario：决策场景注册表（WS5 单一事实源；全局可变，未注册键首调抛 ValueError） ──
from chiyu_kernel.scenario.registry import (
    ScenarioRegistryError, DecisionIOField, IO_KINDS, GATE_KINDS,
    DecisionScenarioProfile, register_scenario,
    get_scenario, require_scenario, list_scenarios, is_registered,
    clear_registry, set_key_format, key_format,
)

# ⛔ memory 内核 experimental 状态（T5′ 写入；消费方不得把本包写进硬判据）
MEMORY_EXPERIMENTAL = True

__all__ = [
    # contract（门面只透出消费方构造 profile 所需的最小面）
    "TenantProfile",
    # store
    "PgVectorStore", "ddl_statements", "vector_index_ddl",
    "assert_no_free_jsonb", "created_schemas", "close_schema", "pad_to_dim",
    "vec_to_pg", "open_store",
    # store 整族 sanctioned 面（机械推导自 store.__all__，见守卫测试）
    "DB_TESTING_ENV", "DSN_ENV", "DSN_TEST_ENV", "FORBIDDEN_HOSTS",
    "FORBIDDEN_ROLES", "IDENT_MAX", "SCHEMA_ENV", "TESTING_FLAG_ENV",
    "VALID_FORMS",
    "DBConfigError", "assert_no_hard_delete", "assert_safe_identifier",
    "close_all", "connect", "cursor", "driver_available", "driver_error",
    "drop_ephemeral_schemas", "drop_test_schema", "ensure_schema",
    "ephemeral_schemas", "get_dsn", "get_schema", "in_testing_context",
    "make_cursor", "mask_dsn", "parse_dsn", "pg_to_vec",
    "probe_schema_isolation", "raw_dsn", "register_ephemeral", "shared",
    "validate_dsn",
    # conn R9：驱动解耦 + 两条硬知识
    "BACKEND_APP_NAME_PREFIX", "DEFAULT_LOCK_TIMEOUT_MS",
    "own_backend_identity", "terminate_backends_sql", "terminate_own_backends",
```

```
    # N5: L0 决策契约常量进 sanctioned 面
    "SPINE_ORDER", "validate_spine_order", "RUBRIC_KEYS", "RUBRIC_SCALE_MAX",
    "EIGHT_ELEMENTS", "AUTONOMOUS_RELEASE_DEFAULT",
    # vector
    "CoverageReport", "compute_coverage", "backfill_plan",
    "DEFAULT_MIN_COVERAGE", "HASH_PROVIDER", "HASH_DIM",
    "InferBackend", "cosine", "hash_signature", "stable_stringify",
    "content_hash", "hash_vector",
    "OnnxRunner", "OnnxUnavailable", "artifact_info", "clear_cache",
    "config_dim", "get_runner", "resolve_model_dir",
    "vector_available", "vector_backend", "vector_info", "vector_is_available",
    "vector_resolve", "is_padded_tag", "tag_for_dim",
    # ontology
    "Particle", "OntologyGraph", "normalize",
    "ensure_embedding", "ensure_tsvector", "ontology_sync", "BackfillChannel",
    # context
    "CONTEXT_LAYERS", "KnowledgeNeed", "SupplyReport", "AssemblyResult",
    "assemble", "LAYER_LABELS_ZH", "KNOWLEDGE_NEEDS_DEFAULT_DIMS",
    "KNOWLEDGE_LAYERS", "LAYER_NAMES", "KnowledgeNeeds", "SupplyDemandGap",
    "build_knowledge_needs", "check_supply_demand", "missing_context_dimensions",
    "MAX_ENTITIES", "MAX_DATASOURCES", "MAX_REQUIRED_FIELDS", "MAX_QUERY_TERMS",
    "MIN_PRECEDENTS", "MIN_EMBEDDING_COVERAGE", "CONTEXT_DIMENSIONS",
    "GAP_MISSING_ENTITY", "GAP_INSUFFICIENT_EXPERIENCE", "GAP_LOW_COVERAGE",
    "GAP_MISSING_DATA", "GAP_MISSING_HITL", "GAP_MISSING_GATE",
    "VTIER_FTS_VECTOR", "VTIER_VECTOR", "VTIER_FTS", "VTIER_NONE", "VTIER_ORDER",
    "TIER_FTS_VECTOR", "TIER_VECTOR", "TIER_FTS", "TIER_NONE", "TIER_ORDER",
    "RecalledItem", "RetrievalEvidence", "RetrievalResult",
    "degraded_recall", "to_supply_layer",
    # actions
    "ActionSpec", "ActionRegistry", "ACTION_MIN", "ACTION_MAX",
    # orchestration
    "MIN_SUCCESS_LEN", "AgentRegistry", "AgentRegistryError", "AgentSpec",
    "TASK_STATES", "TERMINAL_STATES", "BLOCK_KINDS", "TASK_KINDS",
    "SYSTEM_TASK_KINDS", "FAILURE_LIMIT", "MAX_REDO", "ZOMBIE_STALE_SECONDS",
    "SYSTEM_ANCHOR_PREFIX", "ALLOWED_TRANSITIONS", "TaskStateError",
    "is_terminal", "assert_state", "assert_block_kind", "assert_transition",
    "assert_reset", "assert_anchor", "system_anchor", "assert_result_ref",
    "fold_history", "is_stuck", "summarize",
    "DISPATCH_CFG_SECTION", "DISPATCH_CFG_DEFAULTS", "read_dispatch_cfg",
    "dispatch_cfg_report",
    "IDEM_PREFIX", "derive_idem_key", "nightly_idem_key",
    # orchestration runtime
    "Agent", "LiveAgentRegistry", "AgentLoop", "LoopGuard", "StepVerdict",
    # ⛔ 以下 6 项此前**已 import 却未登记 __all__**（2026-10-09 机械比对发现）：
    #   门面 import 了但没导出 ⇒ 消费方 `from api import X` 不可用、只能摸内部路径
    #   （与 R8 四符号同族缺陷）。由 tests/test_facade_import_vs_all.py 机械守住。
    "SharedTask", "SubagentDescriptor", "SubagentPool",
    "TeamMember", "TeamRegistry", "TeamSpec",
    # evolution
    "KnobError", "KnobSpec", "KnobTable", "classify_change",
```

```
    "is_forbidden_change",
    "ReplayVerdict", "shadow_replay", "BASELINE_FIELD",
    "PrescriptionError", "Prescription", "assert_prescription_transition",
    "begin_replay", "apply_atomic", "roll_back",
    "GuardViolation", "READONLY_PATH_PREFIXES", "assert_writable",
    "assert_not_baseline_skill", "g2_suspend_on_redline", "meta_redline_hits",
    "SkillDraftError", "SkillDraft", "validate_draft", "promote",
    "ChannelNotReadyError", "ReadinessReport", "assert_channel_may_open",
    "readiness_report", "CheckResult",
    # events
    "EventBus", "PublishReport", "Subscriber",
    "counting_subscriber", "log_subscriber",
    # audit
    "canonical", "chain_hash", "build_chain", "verify_chain", "VerifyResult",
    "EMPTY_PLACEHOLDER",
    # llm
    "TIERS", "DEFAULT_TIER", "DEFAULT_PARALLEL", "normalize_tier",
    "is_cloud_endpoint", "resolve_storyline_policy", "tier_spec",
    # llm client（同族漏出：已 import 未登记；StubProvider 是消费方离线联调必需）
    "ToolCall", "LlmResponse", "LlmProvider", "StubProvider",
    "DeepSeekProvider", "build_provider",
    # session
    "SessionEvent", "SessionLog", "SessionManager",
    # tools
    "ToolSpec", "ToolRegistry", "ToolNotFound", "ScopeDenied", "GuardDenied",
    "SandboxedFs", "PathEscapeError", "register_fs_tools",
    "CommandRejected", "run_command", "TerminalSession", "register_shell_tools",
    # skills
    "SkillSpec", "SkillRegistry", "SkillConflict", "is_skill_name", "load_catalog",
    # plan/todo
    "TodoItem", "TodoList", "PlanStep", "Plan", "PlanMode", "PlanStateError",
    # mcp bridge
    "McpToolDef", "McpTransport", "InMemoryMcpServer", "McpBridge", "normalize_mcp_name",
    # hooks bus
    "HookBus", "HookResult", "matches",
    # interaction / approval
    "ApprovalGate", "ApprovalRequest",
    # workflow（声明式编排数据模型）
    "WORKFLOW_STEP_STATES", "WorkflowRunId", "WorkflowPhase", "WorkflowMeta",
    "WorkflowStep", "WorkflowResult", "WorkflowStopReason", "WorkflowError",
    "is_fatal_workflow_error", "WorkflowStateError", "Workflow", "WorkflowRunner",
    "register_workflow_tools", "WorkerRun",
    # scenario（可运行场景包；from_dict→scenario_from_dict、
    # build_in_memory_server→scenario_build_in_memory_server，防语义不明裸名。
    # KNOWLEDGE_LAYERS 与 context 层同形同值，门面不重复登记）
    "LEVEL_IDS", "ScenarioPack", "ScenarioPackError",
    "ScenarioStep", "scenario_from_dict", "ScenarioPackStore",
    "ScenarioPackLockDenied", "scenario_build_in_memory_server",
    # scenario 决策场景注册表（WS5：单一事实源；键格式闸可插拔 set_key_format，
    # 支持跨语言消费方命名族共存——2026-10-09 双方核验更正：「根治 CRM 三态键分裂」
```

```
    # 表述作废：config_change ≠ config-change，语义不同必须共存；全局可变，未注册
    # 键首调抛 ValueError）
    "ScenarioRegistryError", "DecisionScenarioProfile", "register_scenario",
    "DecisionIOField", "IO_KINDS", "GATE_KINDS",
    "get_scenario", "require_scenario", "list_scenarios", "is_registered",
    "clear_registry", "set_key_format", "key_format",
    # domain（领域注入 SDK；profile_from_dict/inject_domain 是 L4 域内容进入内核的
    # 统一校验闸，机制非清单）
    "DomainBundle", "DomainInjectionError", "profile_from_dict", "inject_domain",
    # persist（标准 persister：SpineResult→event/particle/记忆网 映射 shim；机制非清单）
    "EVENT_TYPE_DECISION", "DecisionRecord", "judge_worthiness",
    "write_decision_memory_net", "PgVectorStorePersister",
    # persist/B 双轨记忆运行时（LOG append-only + notes upsert + 蒸馏两闸）+ F 白名单
    "DistillReport", "InMemoryMemoryLog", "InMemoryNoteStore", "MemoryLogEntry",
    "MemoryLogStore", "MemoryNote", "NoteStore", "PRESCRIBABLE_PREDICATES",
    "distill", "is_prescribable", "note_hash", "parse_ts",
    # graph（图存储抽象：单源双视图只读派生；协议禁写方法）
    "DECISION_TIME_BASIS", "MAX_TRAVERSAL_HOPS", "TIMELINE_SOURCES",
    "GraphEdge", "TraversalResult", "GraphStore",
    "InMemoryGraphStore", "PgCteGraphStore",
    "relation_view", "timeline_view", "decision_backlinks",
    # mcpserver（MCP 服务端：注册守卫 + 默认拒绝鉴权 + 写操作两阶段确认）
    "RISKS", "McpServerError", "ToolNotExposed", "AuthDenied", "ConfirmRequired",
    "ConfirmTokenInvalid", "ExposedTool", "canonical_args",
    "issue_confirm_token", "verify_confirm_token", "McpServer",
    "StdioTransport", "allow_principals",
    # canary（N7 探针件：决策落库三判据的统一判定式，foundation 单一事实源）
    "CanaryReport", "canary_probe",
    # approval（决策脊柱审批闸 / WS3+B3 三态；与 interaction.ApprovalGate 不同层，命名加 Decision 前缀防静默覆盖）
    "DecisionVerdictState", "DecisionApprovalGate", "DecisionApprovalVerdict",
    "RedlinePriorityGate", "DecisionPassThroughGate",
    # harness native bridge
    "register_native_tools", "native_redline_guard",
    # plugins
    "PluginSpec", "PluginRegistry", "BUILTIN_PLUGINS", "register_builtins",
    "WorkMode", "MODE_PRESETS", "DEFAULT_MODE_ID",
    "mode_ids", "active_plugins", "disabled_plugins", "resolve_work_mode",
    # 状态透出
    "MEMORY_EXPERIMENTAL",
    # N1/R11 版本与能力出口（消费方可用版本断言替代 hasattr 嗅探）
    "API_VERSION", "CONTRACT_VERSION", "CAPABILITIES", "DEPRECATIONS",
    "VALID_STATUSES", "api_version", "capabilities", "capability_status",
    "require_capabilities",
    "contract_version", "deprecations",
    # N1：分发包版本（消费方可版本钉 + 构建期断言），与 pyproject 一致由测试守
    "__version__", "PACKAGE_VERSION",
]

# ⛔ 五处 `vector` 符号在门面加 `vector_` 前缀（`available`→`vector_available`、
#   `backend`→`vector_backend`、`info`→`vector_info`、`resolve`→`vector_resolve`、
```

```
#   `is_available`→`vector_is_available`）。理由是**读性与防静默覆盖**，不是避让
#   现有冲突：门面是唯一公共命名空间，裸名 `resolve`/`info` 在消费方代码里语义
#   不明（"解析什么"）；而门面一旦出现两个同名符号，后者会**静默**覆盖前者
#   （Python 的 import 不报错），与"禁静默中间态"同族。前缀把该风险前置消除。
#   守卫 `test_api_facade_covers_all_layers.py` 用显式改名表核对这 5 个名字。

_store: Optional[PgVectorStore] = None


def open_store(profile: TenantProfile, schema: str = "") -> PgVectorStore:
    """进程级单例（与 demo 同纪律：单实例 + 跨进程锁）。

    ⛔ `schema=""` ⇒ 由 profile/环境决定，内核**不提供**默认域 schema
       （demo 的 `bj` 是该域取值，焊进内核即单域化）。
    """
    global _store
    if _store is None or _store.profile is not profile:
        _store = PgVectorStore(profile=profile, schema=schema)
    return _store
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/approval/__init__.py -----
"""决策脊柱审批闸（WS3 + B3 三态）—— 硬约束先于置信度（A 红线优先 / 反馈闭环金律26）。

⛔ 与 `chiyu_kernel.interaction.ApprovalGate` 是**两个不同层**，职责不重叠、可组合：
   - 本模块 `DecisionApprovalGate`：决策**落库前**的审批裁定（红线/置信度 ⇒ 自动放行 or 转人工）。
   - `interaction.ApprovalGate`：工具**执行前**的人工确认（高危险工具 HITL）。
   命名加 `Decision` 前缀，避免门面与 interaction 同名符号静默覆盖（同 `vector_*` 前缀纪律）。
   组合方式：本闸 `requires_human=True` ⇒ 消费方再走 `interaction.ApprovalGate` 做真人确认。

⛔ B3 三态（方案 Z，2026-10-09）：裁定从两态升三态。
   此前 `requires_human=True`（等人批）与 `redline`（硬拒）共用 `approved=False`，
   脊柱只能一律记 `approval_rejected` —— 「待人工审批」被记成「已驳回」（错值），
   且消费方无法从结果区分"该去催人"还是"该改方案"。
   兼容性：旧消费方读 `.approved` 仍工作（PENDING/REJECTED 下均为 False）；
   `DecisionPassThroughGate` 默认仍 APPROVED（未注入闸的既有消费方零影响）；
   **红线命中恒 REJECTED，不因 pending 豁免**（A 红线优先）。
"""
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum
from typing import Optional, Protocol, Tuple


class DecisionVerdictState(str, Enum):
    """审批裁定三态（B3 方案 Z）。

    - `APPROVED`：可自动落库（脊柱走 persister）；
    - `REJECTED`：硬拒（红线命中），**不可**转 pending、不可自动豁免；
    - `PENDING`：待人工审批（判据缺失 / 未达自主阈值），不落正向边、审计留 `approval_pending`。
    """
```

```
    APPROVED = "approved"
    REJECTED = "rejected"
    PENDING = "pending"


@dataclass(frozen=True)
class DecisionApprovalVerdict:
    """审批裁定（三态）。``approved`` / ``pending`` 为派生属性（旧消费方读 `.approved` 不破）。"""

    state: DecisionVerdictState = DecisionVerdictState.APPROVED
    reason: str = ""
    requires_human: bool = False

    @property
    def approved(self) -> bool:
        """兼容字段：仅 APPROVED 为真（旧两态消费方读此属性不破）。"""
        return self.state is DecisionVerdictState.APPROVED

    @property
    def pending(self) -> bool:
        """兼容字段：PENDING 为真（供消费方区分"待人工"与"已驳回"）。"""
        return self.state is DecisionVerdictState.PENDING

    @property
    def rejected(self) -> bool:
        """兼容字段：REJECTED 为真。"""
        return self.state is DecisionVerdictState.REJECTED


class DecisionApprovalGate(Protocol):
    """决策脊柱 S4→S5 审批闸协议（鸭子类型，消费方可注入自有实现）。"""

    def decide(self, *, decision_id: str, confidence: float, redline_hit: bool,
               redline_reasons: Tuple[str, ...], autonomous: bool,
               rubric_missing: Tuple[str, ...], elements_missing: Tuple[str, ...],
               threshold: float) -> DecisionApprovalVerdict:
        ...


@dataclass
class RedlinePriorityGate:
    """默认审批闸 —— embody A 红线优先 + 硬约束先于置信度（金律26）。

    裁定顺序（不可重排，顺序即语义）：
      1. 红线命中 → **REJECTED**（不因置信度高而豁免，也不得降级为 pending）。
      2. 判据缺失（九尺子/八要素）→ **PENDING**（转人工确认，不自动放行）。
      3. ``autonomous`` 为真 ⇒ APPROVED；否则 PENDING（未达自主阈值，等人批）。
    """
```

```
    def decide(self, *, decision_id: str, confidence: float, redline_hit: bool,
               redline_reasons: Tuple[str, ...], autonomous: bool,
               rubric_missing: Tuple[str, ...], elements_missing: Tuple[str, ...],
               threshold: float) -> DecisionApprovalVerdict:
        # 1. 硬约束先于置信度：红线命中立即硬拒，高置信度不豁免（A 红线优先 / 金律26）
        if redline_hit:
            reason = "红线命中（硬约束优先于置信度）"
            if redline_reasons:
                reason += "：" + "；".join(redline_reasons)
            return DecisionApprovalVerdict(
                state=DecisionVerdictState.REJECTED, reason=reason, requires_human=True)
        # 2. 判据缺失 → 待人工确认（B3：等人批 ≠ 已驳回）
        missing = tuple(list(rubric_missing) + list(elements_missing))
        if missing:
            return DecisionApprovalVerdict(
                state=DecisionVerdictState.PENDING,
                reason="判据缺失需人工确认：" + "、".join(missing),
                requires_human=True)
        # 3. autonomous 为真（红线未命中且置信度达门槛）⇒ 自动放行；否则待人工
        if autonomous:
            return DecisionApprovalVerdict(
                state=DecisionVerdictState.APPROVED, reason="置信度达标，自动放行")
        return DecisionApprovalVerdict(
            state=DecisionVerdictState.PENDING,
            reason=f"未达自主放行条件（置信度 {confidence:.3f}），需人工确认",
            requires_human=True)


@dataclass
class DecisionPassThroughGate:
    """审批闸透传（默认）：不拦截，等价于未注入审批闸（保留 run_spine 既有行为）。"""

    def decide(self, *, decision_id: str, confidence: float, redline_hit: bool,
               redline_reasons: Tuple[str, ...], autonomous: bool,
               rubric_missing: Tuple[str, ...], elements_missing: Tuple[str, ...],
               threshold: float) -> DecisionApprovalVerdict:
        return DecisionApprovalVerdict(
            state=DecisionVerdictState.APPROVED, reason="无审批闸（透传）")


__all__ = [
    "DecisionVerdictState", "DecisionApprovalVerdict", "DecisionApprovalGate",
    "RedlinePriorityGate", "DecisionPassThroughGate",
]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/audit/__init__.py -----
"""审计子系统 —— SHA-256 链式审计（SKILL #10 机制层；★ 内核新建，demo 无此目录）。"""
from .chain import (
    EMPTY_PLACEHOLDER,
    VerifyResult,
    build_chain,
```

```
    canonical,
    chain_hash,
    verify_chain,
)

__all__ = ["EMPTY_PLACEHOLDER", "VerifyResult", "build_chain",
           "canonical", "chain_hash", "verify_chain"]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/audit/chain.py -----
"""SHA-256 链式审计（SKILL #10 的机制层）—— ★ 内核新建，demo 无此目录。

⛔ 为什么是链式而不是逐条哈希
──────────────────────────────────────────
逐条哈希只能证明"这条没被改"；链式还能证明"**没有中间记录被删掉**"——
后者才是审计被绕过的常见形式（删掉不利记录，其余保持自洽）。

    h_i = sha256( h_{i-1} + canonical(record_i) ), h_0 = "0"*64

canonical 规则：按 AUDIT_REQUIRED_FIELDS 顺序取字段，缺失即 "∅"（占位符而非跳过
—— 跳过会让"删字段"与"原值就是空"产生同一哈希，蓄意者可借此抹除字段）。
"""
from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass
from typing import Iterable, List, Mapping, Sequence

from chiyu_kernel.contract import (
    AUDIT_GENESIS_HASH,
    AUDIT_HASH_ALGO,
    AUDIT_REQUIRED_FIELDS,
)

EMPTY_PLACEHOLDER = "∅"


def canonical(record: Mapping, fields: Sequence[str] = AUDIT_REQUIRED_FIELDS) -> str:
    payload = {f: record.get(f, EMPTY_PLACEHOLDER) for f in fields}
    return json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":"))


def chain_hash(prev_hash: str, record: Mapping) -> str:
    if AUDIT_HASH_ALGO != "sha256":
        raise ValueError(f"未支持的审计哈希算法：{AUDIT_HASH_ALGO}")
    h = hashlib.sha256()
    h.update(prev_hash.encode("utf-8"))
    h.update(canonical(record).encode("utf-8"))
    return h.hexdigest()
```

```
def build_chain(records: Iterable[Mapping]) -> List[str]:
    prev = AUDIT_GENESIS_HASH
    out: List[str] = []
    for r in records:
        prev = chain_hash(prev, r)
        out.append(prev)
    return out


@dataclass(frozen=True)
class VerifyResult:
    ok: bool
    checked: int
    first_bad_index: int = -1
    reason: str = ""


def verify_chain(records: Sequence[Mapping], expected: Sequence[str]) -> VerifyResult:
    """重算并比对。返回首个失配位置（-1 表示全链通过）。"""
    if len(records) != len(expected):
        return VerifyResult(False, min(len(records), len(expected)), -1,
                            f"长度不一致：records={len(records)} expected={len(expected)}")
    prev = AUDIT_GENESIS_HASH
    for i, r in enumerate(records):
        prev = chain_hash(prev, r)
        if prev != expected[i]:
            return VerifyResult(False, i + 1, i, f"第 {i} 条哈希失配")
    return VerifyResult(True, len(records))


__all__ = ["canonical", "chain_hash", "build_chain", "verify_chain",
           "VerifyResult", "EMPTY_PLACEHOLDER"]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/canary/__init__.py -----
"""canary 探针件（N7）—— 决策落库三判据的**统一判定式**（foundation 单一事实源）。

⛔ 为什么要在基座 ship 这个：三个消费方各自自造探针 ⇒ 判据口径漂移
   （"count>0 就算落库"这类恒绿探针曾是本仓真实事故）。本模块把三判据
   的**判定式**收归基座维护，消费方只注入接线（store + 决策回调）。

三判据（缺一不可，任一不满足即探针失败）：
  1. **决策真落库**：event 行存在 **且** particle 行存在（只查 event 不查
     particle 挡不住"半写"—— 两张表须同事务成对出现）；
  2. **记忆网真物化**：`graph_edge` 中该决策的 `edge_source='engine'` 边数
     **恰好等于**注入 references 的非空谓词数（多写=幻影边，少写=漏物化）；
  3. **红线挡落库**：红线决策回调的结果不落库且被审批闸拦下
     （REJECTED 或 PENDING；注入了红线回调才可判定，否则该判据为 False）。

接口说明（与 enterprise 反馈 #N7 原案 `canary_probe(dsn, profile)` 的差异，诚实标注）：
  取 `store`（鸭子类型：`execute` / `execute_fetch` / `schema`）而非 `dsn` ——
  消费方（如 APS 用 psycopg3）已持有自己的连接层，注入 store 免内核绑死驱动
```

```
  （与 R9 驱动解耦同一取向）；dsn 场景由消费方先 `PgVectorStore(profile)` 自建。
  `run_decision` / `run_redline_decision` 为消费方接线的决策回调
  （内核不得反向依赖 chiyu_decision，分层铁律）。
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Any, Callable, Mapping, Optional, Sequence

__all__ = ["CanaryReport", "canary_probe"]


@dataclass(frozen=True)
class CanaryReport:
    """探针结论：三判据逐项布尔 + 细节（排障用，不含敏感值）。"""

    decision_persisted: bool
    memory_net_materialized: bool
    redline_blocked: bool
    details: Mapping[str, Any] = field(default_factory=dict)

    @property
    def ok(self) -> bool:
        """三判据全绿才算通过（部分绿 = 未通过，禁"主要判据绿"式放水）。"""
        return self.decision_persisted and self.memory_net_materialized \
            and self.redline_blocked


def _q(schema: str, table: str) -> str:
    return f"{schema}.{table}" if schema else table


def _table_of(sql: str, schema: str, table: str) -> bool:
    """判定 SQL 是否查询 `<schema>.<table>`（供打桩 store 路由读请求）。"""
    pat = re.compile(rf"FROM\s+{re.escape(_q(schema, table))}\b", re.IGNORECASE)
    return bool(pat.search(sql))


def canary_probe(*, store: Any,
                 run_decision: Callable[[], Any],
                 decision_id: str = "canary-dec-1",
                 references: Optional[Mapping[str, Sequence[str]]] = None,
                 run_redline_decision: Optional[Callable[[], Any]] = None) -> CanaryReport:
    """对已接线的决策链路执行三判据。**只读核验**，自身不写库。

    参数：
      store               消费方 store（PgVectorStore 或同鸭子类型；须有
                          `schema` / `execute_fetch`）
      run_decision        消费方决策回调：执行一次**正常**决策
```

```
                      （run_spine + persister 已接到同一 store），返回
                      SpineResult 同形对象（须有 `persisted` 属性）
  decision_id         本轮回调使用的 decision_id（须与回调内一致）
  references          记忆网供给（键=谓词、值=端点列表）；None/空 ⇒
                      判据 2 不可判定（False，禁静默放行）
  run_redline_decision 红线决策回调：执行一次**红线命中**的决策；
                      None ⇒ 判据 3 不可判定（False）
"""
schema = getattr(store, "schema", "")
details: dict = {}

# ——— 判据 1：决策真落库（event + particle 成对）————————————————————————————
result = run_decision()
details["run_persisted"] = bool(getattr(result, "persisted", False))
event_row = store.execute_fetch(
    f"SELECT id FROM {_q(schema, 'event')} WHERE id=%s", (decision_id,))
particle_row = store.execute_fetch(
    f"SELECT particle_id FROM {_q(schema, 'particle')} WHERE particle_id=%s",
    (decision_id,))
details["event_row"] = event_row is not None
details["particle_row"] = particle_row is not None
decision_persisted = (details["run_persisted"] and details["event_row"]
                      and details["particle_row"])

# ——— 判据 2：记忆网真物化（engine 边数 == 非空谓词数，恰好相等）——————————
expected_edges = sum(1 for v in (references or {}).values() if v) \
    if references else 0
if expected_edges == 0:
    details["memory_net"] = "未提供 references ⇒ 判据 2 不可判定"
    memory_net_materialized = False
else:
    row = store.execute_fetch(
        f"SELECT count(*) AS n FROM {_q(schema, 'graph_edge')} "
        f"WHERE from_id=%s AND edge_source='engine'", (decision_id,))
    actual = int(row["n"]) if isinstance(row, dict) else int(row[0])
    details["memory_net"] = f"engine 边 {actual} / 期望 {expected_edges}"
    memory_net_materialized = (actual == expected_edges)

# ——— 判据 3：红线挡落库 ——————————————————————————————————————————————————
if run_redline_decision is None:
    details["redline"] = "未注入红线决策回调 ⇒ 判据 3 不可判定"
    redline_blocked = False
else:
    r2 = run_redline_decision()
    rejected = getattr(r2, "approval_rejected", None) is not None
    pending = bool(getattr(r2, "pending", False))
    not_persisted = not getattr(r2, "persisted", False)
    details["redline"] = (f"persisted={getattr(r2, 'persisted', None)}, "
                          f"rejected={rejected}, pending={pending}")
    redline_blocked = not_persisted and (rejected or pending)
```

```
    return CanaryReport(
        decision_persisted=decision_persisted,
        memory_net_materialized=memory_net_materialized,
        redline_blocked=redline_blocked,
        details=details,
    )
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/context/__init__.py -----
"""上下文层 —— 需求侧编码 + L1-L4 装配 + 三级降级召回（L0/L1）。

抽取自 demo `context/{knowledge_needs,retrieval}.py` + 新增 `assembly.py`
（SKILL #4 上下文分层的机制化，2026-09-21，P1 T4）。
⛔ `timeline.py` 不搬迁 —— 属 demo 应用层（群诉时间线语义），内核只提供
`assembly` 这一**机制**。
"""
from .assembly import (
    AssemblyResult,
    CONTEXT_LAYERS,
    KNOWLEDGE_NEEDS_DEFAULT_DIMS,
    KnowledgeNeed,
    LAYER_LABELS_ZH,
    SupplyReport,
    assemble,
)
from .knowledge_needs import (
    MAX_DATASOURCES,
    MAX_ENTITIES,
    MAX_QUERY_TERMS,
    MAX_REQUIRED_FIELDS,
    MIN_EMBEDDING_COVERAGE,
    MIN_PRECEDENTS,
    CONTEXT_DIMENSIONS,
    GAP_INSUFFICIENT_EXPERIENCE,
    GAP_LOW_COVERAGE,
    GAP_MISSING_DATA,
    GAP_MISSING_ENTITY,
    GAP_MISSING_GATE,
    GAP_MISSING_HITL,
    KnowledgeNeeds,
    KNOWLEDGE_LAYERS,
    LAYER_NAMES,
    SupplyDemandGap,
    build_knowledge_needs,
    check_supply_demand,
    missing_context_dimensions,
)
from .retrieval import (
    RecalledItem,
    RetrievalEvidence,
    RetrievalResult,
```

```
    TIER_FTS,
    TIER_FTS_VECTOR,
    TIER_NONE,
    TIER_ORDER,
    TIER_VECTOR,
    VTIER_FTS,
    VTIER_FTS_VECTOR,
    VTIER_NONE,
    VTIER_ORDER,
    VTIER_VECTOR,
    degraded_recall,
    to_supply_layer,
)

__all__ = [
    # assembly
    "CONTEXT_LAYERS", "KnowledgeNeed", "SupplyReport", "AssemblyResult",
    "assemble", "LAYER_LABELS_ZH", "KNOWLEDGE_NEEDS_DEFAULT_DIMS",
    # knowledge_needs
    "KNOWLEDGE_LAYERS", "LAYER_NAMES", "KnowledgeNeeds", "SupplyDemandGap",
    "build_knowledge_needs", "check_supply_demand", "missing_context_dimensions",
    "MAX_ENTITIES", "MAX_DATASOURCES", "MAX_REQUIRED_FIELDS", "MAX_QUERY_TERMS",
    "MIN_PRECEDENTS", "MIN_EMBEDDING_COVERAGE", "CONTEXT_DIMENSIONS",
    "GAP_MISSING_ENTITY", "GAP_INSUFFICIENT_EXPERIENCE", "GAP_LOW_COVERAGE",
    "GAP_MISSING_DATA", "GAP_MISSING_HITL", "GAP_MISSING_GATE",
    # retrieval
    "VTIER_FTS_VECTOR", "VTIER_VECTOR", "VTIER_FTS", "VTIER_NONE", "VTIER_ORDER",
    "TIER_FTS_VECTOR", "TIER_VECTOR", "TIER_FTS", "TIER_NONE", "TIER_ORDER",
    "RecalledItem", "RetrievalEvidence", "RetrievalResult",
    "degraded_recall", "to_supply_layer",
]
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/context/assembly.py -----
"""上下文装配（L1 – L4）+ KnowledgeNeeds 供需校验（★ 新增，demo 未收敛）。

四层语义（SKILL #4）：
    L1 知识底座      静态领域知识（本体 / 受控词表 / 规格）
    L2 历史决策      先例（含**反面先例** —— 第 9 尺 fairmess 的输入）
    L3 执行协同      任务态 / 派发 / 编排上下文
    L4 治理决策      红线 / 审批 / 权限 / 合规

⛔ 供需一致性校验（为什么必须可失败）
——————————————————————————————————————————————————————
第 9 尺的 relevance 尺算的是「所需维度 ∩ 已供给维度」。若供给表能静默缺少
某层（例如反例只在"检索到了"但从未"被消费"），relevance 会在**看起来正常**的
情况下恒得低分——这正是 demo 里"评分被降级为叙述留痕"的同族缺陷。
故 assemble() 返回 `supply_report`，缺口以**结构化字段**给出，调用方必须显式处理。

⛔ 零领域词：本模块只声明机制，不声明任何领域取值（无"群诉/工单/粒子"字面量）。
"""

from __future__ import annotations
```

```
from dataclasses import dataclass, field
from typing import Iterable, Mapping, Sequence

# 四层语义（与 demo knowledge_needs.KNOWLEDGE_LAYERS 对齐的机制常量）
CONTEXT_LAYERS = ("L1", "L2", "L3", "L4")

LAYER_LABELS_ZH = {
    "L1": "知识底座", "L2": "历史决策", "L3": "执行协同", "L4": "治理决策",
}

# 默认供给维度（机制默认：dim 键缺省时按此匹配；消费方可覆盖）
KNOWLEDGE_NEEDS_DEFAULT_DIMS = ("semantics", "precedent", "operation", "governance")


@dataclass(frozen=True)
class KnowledgeNeed:
    """需求侧编码：本决策需要哪一层、哪几个维度、哪个键。"""

    layer: str
    dims: tuple = ()
    key: str = ""

    def __post_init__(self) -> None:
        if self.layer not in CONTEXT_LAYERS:
            raise ValueError(f"layer 须 ∈ {CONTEXT_LAYERS}：{self.layer!r}")


@dataclass
class SupplyReport:
    required: tuple = ()
    supplied: tuple = ()
    missing: tuple = ()
    degraded_layers: tuple = ()

    @property
    def consistent(self) -> bool:
        return not self.missing

    def as_metrics(self) -> Mapping[str, float]:
        req = len(self.required)
        hit = req - len(self.missing)
        return {
            "context_supply_rate": round(hit / req, 4) if req else 1.0,
            "context_missing": float(len(self.missing)),
            "context_degraded_layers": float(len(self.degraded_layers)),
        }


@dataclass
```

```
class AssemblyResult:
    facts: dict = field(default_factory=dict)
    supply_report: SupplyReport = field(default_factory=SupplyReport)


def assemble(needs: Sequence[KnowledgeNeed],
             provided: Mapping[str, Iterable]) -> AssemblyResult:
    """按需装配。

    needs    KnowledgeNeed 序列（需求侧，由调用方按场景声明）
    provided {layer: [{"dim":..., "key":..., "value":...}, ...]}（供给侧）
    返回    facts + supply_report（缺口**结构化**返回，不吞不糊）
    """
    facts: dict = {}
    supplied_keys: list = []
    missing: list = []
    degraded: list = []
    for need in needs:
        rows = list(provided.get(need.layer) or [])
        if not rows:
            degraded.append(need.layer)
        hit = [r for r in rows
               if (not need.dims or _dim_of(r) in need.dims)
               and (not need.key or r.get("key") == need.key)]
        if hit:
            supplied_keys.append((need.layer, need.key))
            for r in hit:
                facts.setdefault(need.layer, []).append(r.get("value"))
        else:
            missing.append((need.layer, need.key))
    return AssemblyResult(
        facts=facts,
        supply_report=SupplyReport(
            required=tuple((n.layer, n.key) for n in needs),
            supplied=tuple(supplied_keys),
            missing=tuple(missing),
            degraded_layers=tuple(degraded),
        ),
    )


def _dim_of(row: Mapping) -> str:
    d = row.get("dim")
    if isinstance(d, dict):
        return d.get("key") or d.get("dim") or ""
    return d or ""


__all__ = ["CONTEXT_LAYERS", "LAYER_LABELS_ZH", "KNOWLEDGE_NEEDS_DEFAULT_DIMS",
           "KnowledgeNeed", "SupplyReport", "AssemblyResult", "assemble"]
```

```
# ----- FILE: packages/chiyu-kernel/src/chiyu_kernel/context/knowledge_needs.py -----
"""需求侧编码 + 供需缺口判定（SKILL #4 —— 内核版，去领域化）。

抽取自 demo `context/knowledge_needs.py`（2026-09-21，P1 T4）。
⛔ 去领域化的三处精确化（源码核实）：
   1. demo `REQUIRED_FIELDS_8`（text_sim/…/cluster_size…）与
      `DEFAULT_DATA_SOURCES`（candidates/cluster_profile…）含**领域字面量**
      （cluster/place 类），属 L4 领域取值 ⇒ 全部参数化，调用方显式传；
      本模块只保留**机制**（上限约束、缺口判据、七维齐全轴）。
   2. 返回类型 → `KnowledgeNeeds`（与 assembly.KnowledgeNeed 的对接经\n      `to_assembly_needs()`）。
   3. `check_supply_demand` 的 `supply` 契约键（entity_links/historical_cases/\n      embedding_coverage/data_rows/hitl_checkpoints/gates）为**机制层判据**，

四层模型：L1 知识底座 / L2 历史决策 / L3 执行协同 / L4 治理决策。
缺口类型：MISSING_ENTITY / INSUFFICIENT_EXPERIENCE / LOW_COVERAGE /\nMISSING_DATA / MISSING_HITL / MISSING_GATE。
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Iterable, Mapping, Sequence

from .assembly import CONTEXT_LAYERS, KnowledgeNeed

# ——— 四层模型（机制常量）————————————————————————————————————————————————————————
KNOWLEDGE_LAYERS: tuple[str, ...] = CONTEXT_LAYERS

LAYER_NAMES: dict[str, str] = {
    "L1": "知识底座",   # 实体/本体/字典（图谱一跳 + 实体向量）
    "L2": "历史决策",   # 先例/经验（向量+全文 RRF 召回）
    "L3": "执行协同",   # HITL/审批（结构化直查，无 I/O 失败面）
    "L4": "治理决策",   # 红线/闸门（实时查询，禁缓存）
}

# ——— 上限（对齐 SKILL #4 第 2 轮编码约束，机制级）————————————————————————————————
MAX_ENTITIES: int = 24
MAX_DATASOURCES: int = 16
MAX_REQUIRED_FIELDS: int = 40
MAX_QUERY_TERMS: int = 20

# ——— 判据阈值（对齐 SKILL #4 §5.2 规则矩阵，机制级）————————————————————————————————
MIN_PRECEDENTS: int = 3              # level>=2 → historicalCases >= 3
MIN_EMBEDDING_COVERAGE: float = 0.80  # level>=2 → 向量覆盖率 >= 80%

# ——— 七维（齐全轴，与深度轴正交；机制级，无领域词）————————————————————————————————
CONTEXT_DIMENSIONS: tuple[str, ...] = (
    "identity", "structure", "semantics", "time_config",
    "decision_history", "operational_state", "governance",
)

# ——— 缺口类型（机制级）————————————————————————————————————————————————————————
GAP_MISSING_ENTITY = "MISSING_ENTITY"
```

```
GAP_INSUFFICIENT_EXPERIENCE = "INSUFFICIENT_EXPERIENCE"
GAP_LOW_COVERAGE = "LOW_COVERAGE"
GAP_MISSING_DATA = "MISSING_DATA"
GAP_MISSING_HITL = "MISSING_HITL"
GAP_MISSING_GATE = "MISSING_GATE"


def _clamp_unique(items: Iterable[str], limit: int) -> tuple[str, ...]:
    """去重 + 保序 + 截断（空串丢弃）。需求编码一律走此入口，防「无声膨胀」。"""
    seen: set[str] = set()
    out: list[str] = []
    for raw in items:
        s = str(raw).strip()
        if not s or s in seen:
            continue
        seen.add(s)
        out.append(s)
        if len(out) >= limit:
            break
    return tuple(out)


@dataclass(frozen=True)
class KnowledgeNeeds:
    """需求侧编码：本次决策任务**需要**哪些知识。"""

    task_name: str
    knowledge_level: int
    semantic_entities: tuple[str, ...] = ()
    data_sources: tuple[str, ...] = ()
    required_fields: tuple[str, ...] = ()
    query_terms: tuple[str, ...] = ()

    @property
    def active_layers(self) -> tuple[str, ...]:
        """累积语义：level=x → 注入 L1..Lx（对齐 SKILL #4「level≥1 → L1」）。"""
        lv = max(0, min(int(self.knowledge_level), len(KNOWLEDGE_LAYERS)))
        return KNOWLEDGE_LAYERS[:lv]

    @property
    def semantic_entity_limit(self) -> int:
        return MAX_ENTITIES

    def to_assembly_needs(self, *, dims_by_layer: Mapping[str, Sequence[str]] | None = None,
                          keys_by_layer: Mapping[str, str] | None = None) -> tuple[KnowledgeNeed, ...]:
        """（+T4 对接）转为 `assembly.KnowledgeNeed` 序列，供 `assemble()` 消费。

        dims_by_layer / keys_by_layer 缺省时用机制默认（dims=()、key=""），
        即「该层有供给即计入」。调用方可按场景细化。
        """
```

```
        dims = dims_by_layer or {}
        keys = keys_by_layer or {}
        return tuple(
            KnowledgeNeed(layer=lv,
                          dims=tuple(dims.get(lv, ())),
                          key=keys.get(lv, ""))
            for lv in self.active_layers
        )

    def to_dict(self) -> dict[str, Any]:
        return {
            "task_name": self.task_name,
            "knowledge_level": self.knowledge_level,
            "semanticEntities": list(self.semantic_entities),
            "dataSources": list(self.data_sources),
            "requiredFields": list(self.required_fields),
            "queryTerms": list(self.query_terms),
            "activeLayers": list(self.active_layers),
        }


@dataclass(frozen=True)
class SupplyDemandGap:
    """供需缺口：显式标注（不静默吞）。"""

    layer: str
    code: str
    detail: str

    def to_dict(self) -> dict[str, str]:
        return {"layer": self.layer, "code": self.code, "detail": self.detail}


def build_knowledge_needs(
    task_name: str,
    knowledge_level: int = 2,
    *,
    entities: Sequence[str] = (),
    places: Sequence[str] = (),
    categories: Sequence[str] = (),
    data_sources: Sequence[str] = (),
    required_fields: Sequence[str] = (),
    query_terms: Sequence[str] = (),
) -> KnowledgeNeeds:
    """四段解析（对齐 SKILL #4 第 2 轮）：种子 / 源 / 字段 / 查询词。

    语义实体 = ENTITY + PLACE + CATEGORY 合并去重（图种子，≤24）——\n    参数名 `places`/`categories` 是**调用方传入的领域取值**，本模块只做合并去重。\n    """
    terms = [t for t in query_terms if "_" not in str(t)]
    return KnowledgeNeeds(
```

```
        task_name=str(task_name),
        knowledge_level=int(knowledge_level),
        semantic_entities=_clamp_unique(
            [*entities, *places, *categories], MAX_ENTITIES),
        data_sources=_clamp_unique(data_sources, MAX_DATASOURCES),
        required_fields=_clamp_unique(required_fields, MAX_REQUIRED_FIELDS),
        query_terms=_clamp_unique(terms, MAX_QUERY_TERMS),
    )


def _as_int(v: Any, default: int = 0) -> int:
    try:
        return int(v)
    except (TypeError, ValueError):
        return default


def check_supply_demand(
    needs: KnowledgeNeeds,
    supply: Mapping[str, Any],
) -> list[SupplyDemandGap]:
    """供需一致性校验（SKILL #4 §5.2 规则矩阵，机制级翻译）。

    `supply` 契约（缺键 = 该层无供给 ⇒ 触发对应缺口）：
      - entity_links: Mapping[str, int]    语义实体 → 图谱关联数
      - historical_cases: int              可用先例数
      - embedding_coverage: float          向量覆盖率 0..1
      - data_rows: Mapping[str, int]       数据源 → 行数
      - hitl_checkpoints: Sequence         人工池待办
      - gates: Mapping[str, Any]           红线/闸门状态

    返回**缺口列表**（空 = 配齐）。任一缺口必须由调用方写入 trace，禁止吞。
    """
    gaps: list[SupplyDemandGap] = []
    layers = set(needs.active_layers)

    # L1：每个语义实体须在图谱有 ≥1 关联
    if "L1" in layers and needs.semantic_entities:
        links = supply.get("entity_links") or {}
        orphaned = [e for e in needs.semantic_entities if _as_int(links.get(e)) < 1]
        if orphaned:
            gaps.append(SupplyDemandGap(
                "L1", GAP_MISSING_ENTITY,
                f"{len(orphaned)}/{len(needs.semantic_entities)} 个强实体未入图："
                f"{orphaned[:5]}"))

    # L2：先例数 + 向量覆盖率
    if "L2" in layers:
        cases = _as_int(supply.get("historical_cases"))
        if cases < MIN_PRECEDENTS:
```

```
    "SCHEMA_VERSION", "PAGE_TYPES", "COMPONENT_KINDS", "THEMES", "FILTER_OPS",
    "AGG_FUNCS", "DEFAULT_ALLOWED_ACTIONS", "GuardResult", "guard_nl_input",
    "ParseResult", "parse_nl_to_schema", "RenderResult", "render_page",
    "ValidationError", "validate_page_schema", "CreateResult", "PageDraft", "PageStore",
    "onboarding_steps", "mode_cards", "level_cards", "workflow_cards",
    "direction_cards", "render_onboarding", "OnboardingResult",
]
# ----- FILE: packages/chiyu-portal/src/chiyu_portal/api.py -----
"""chiyu-portal 唯一公开门面（T5 收口 • 完整导出）。

三段式：NL → 受控 Schema → 运行时渲染器。渲染器是唯一渲染出口，Schema 是唯一输入。
"""
from __future__ import annotations

from .schema import (
    SCHEMA_VERSION, PAGE_TYPES, COMPONENT_KINDS, THEMES, FILTER_OPS,
    AGG_FUNCS, DEFAULT_ALLOWED_ACTIONS, ValidationError, validate_page_schema,
)
from .guardrails import GuardResult, guard_nl_input
from .nl_parser import ParseResult, parse_nl_to_schema
from .renderer import RenderResult, render_page, render_conversation
from .page_store import CreateResult, PageDraft, PageStore
from .onboarding import (
    OnboardingResult, mode_cards, level_cards, workflow_cards,
    direction_cards, onboarding_steps, render_onboarding,
)

__all__ = [
    "SCHEMA_VERSION", "PAGE_TYPES", "COMPONENT_KINDS", "THEMES", "FILTER_OPS",
    "AGG_FUNCS", "DEFAULT_ALLOWED_ACTIONS",
    "GuardResult", "guard_nl_input",
    "ParseResult", "parse_nl_to_schema",
    "RenderResult", "render_page", "render_conversation",
    "ValidationError", "validate_page_schema",
    "CreateResult", "PageDraft", "PageStore",
    "onboarding_steps", "mode_cards", "level_cards", "workflow_cards",
    "direction_cards", "render_onboarding", "OnboardingResult",
]
# ----- FILE: packages/chiyu-portal/src/chiyu_portal/guardrails.py -----
"""输入层护栏：拦截 NL 中的脚本/注入，从源头杜绝「NL 直出 HTML」。

纵深防御第一层（输入层）。第二层结构层 `validate_page_schema`，第三层渲染层 `render_page`。
"""
from __future__ import annotations

import re
from dataclasses import dataclass

# 注入模式（IGNORECASE）：脚本 / 伪协议 / 事件属性 / 动态求值 / 嵌入对象
_INJECTION_PATTERNS: tuple[re.Pattern[str], ...] = (
```

```
    re.compile(r"<\s*script", re.IGNORECASE),
    re.compile(r"javascript\s*:", re.IGNORECASE),
    re.compile(r"<\s*iframe", re.IGNORECASE),
    re.compile(r"<\s*object", re.IGNORECASE),
    re.compile(r"<\s*embed", re.IGNORECASE),
    re.compile(r"on\w+\s*=", re.IGNORECASE),
    re.compile(r"eval\s*\("),
)


@dataclass(frozen=True)
class GuardResult:
    """输入护栏结果。``ok`` 决定可否进入解析层。"""

    ok: bool
    reason: str = ""

    def __bool__(self) -> bool:
        return self.ok


def guard_nl_input(text: str) -> GuardResult:
    """拦截脚本注入。命中任一模式即拒绝，返回字段级原因。"""
    if not isinstance(text, str):
        return GuardResult(False, "输入必须是字符串")
    for pat in _INJECTION_PATTERNS:
        m = pat.search(text)
        if m:
            return GuardResult(False, f"检测到注入模式: {m.group(0)!r}")
    return GuardResult(True)


__all__ = ["GuardResult", "guard_nl_input"]
# ----- FILE: packages/chiyu-portal/src/chiyu_portal/nl_parser.py -----
"""NL 解析层：自然语言 → 受控 Schema。⛔ 绝不产出 HTML（守卫 + 测试双重锁死）。

确定性规则映射（无 LLM 直出）。实体→粒子经注入 ``entity_map``；未识别实体 → 低置信 +
``needsClarification``。三段式中本层只产出 Schema，渲染器的 HTML 是另一层的事。
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Any

from .schema import SCHEMA_VERSION

# 意图词 → 页面类型（缺省 dashboard）。顺序即优先级。
_TYPE_RULES: tuple[tuple[tuple[str, ...], str], ...] = (
    (("工作台", "协作", "workspace"), "workspace"),
```

```
    (("审批", "审核", "approval"), "approval"),
    (("表单", "录入", "编辑", "form"), "form"),
    (("详情", "明细", "detail"), "detail"),
    (("表", "列表", "清单", "table"), "table"),
    (("看板", "dashboard", "指标", "总览", "概览"), "dashboard"),
)


@dataclass
class ParseResult:
    """解析结果：结构化 Schema（绝不 HTML）+ 置信度 + 是否需澄清。"""

    schema: dict[str, Any]
    confidence: float
    needs_clarification: bool
    notes: list[str] = field(default_factory=list)

    def to_dict(self) -> dict[str, Any]:
        return {
            "schema": self.schema,
            "confidence": self.confidence,
            "needsClarification": self.needs_clarification,
            "notes": self.notes,
        }


def parse_nl_to_schema(
    text: str,
    *,
    entity_map: dict[str, str] | None = None,
    title: str | None = None,
) -> ParseResult:
    """将自然语言描述解析为受控 Schema（绝不直出 HTML）。

    ``entity_map``：NL 实体词 → 粒子类型（消费方注入）。未命中 → 降置信 + needsClarification。
    """
    entity_map = entity_map or {}
    notes: list[str] = []

    schema_type = "dashboard"
    for keywords, t in _TYPE_RULES:
        if any(k in text for k in keywords):
            schema_type = t
            break

    particle_type: str | None = None
    for token, pt in entity_map.items():
        if token and token in text:
            particle_type = pt
            break
```

```
    confidence = 0.9
    needs = False
    if particle_type is None:
        confidence = 0.3
        needs = True
        notes.append("未识别实体，无法确定粒子类型")

    highlight = _detect_threshold(text)

    components: list[dict[str, Any]] = []
    if particle_type is not None:
        metric: dict[str, Any] = {
            "field": "value", "agg": "count", "label": "计数",
        }
        comp: dict[str, Any] = {
            "kind": "metric-card",
            "title": title or f"{schema_type} 视图",
            "dataBinding": {
                "source": "particle",
                "particleType": particle_type,
                "metrics": [metric],
            },
        }
        if highlight is not None:
            comp["style"] = {"highlight": highlight}
        components.append(comp)

    schema: dict[str, Any] = {
        "version": SCHEMA_VERSION,
        "type": schema_type,
        "title": title or f"{schema_type} 视图",
        "layout": {"columns": 2, "theme": "light"},
        "components": components,
    }
    return ParseResult(
        schema=schema, confidence=confidence,
        needs_clarification=needs, notes=notes,
    )


def _detect_threshold(text: str) -> dict[str, Any] | None:
    """阈值高亮：低于|小于|少于 X% → lt；高于|超过|大于|多于 X% → gt（红色）。"""
    m = re.search(r"(低于|小于|少于)\s*(\d+(?:\.\d+)?)\s*%?", text)
    if m:
        return {"field": "value", "op": "lt", "value": float(m.group(2)) / 100.0, "color": "red"}
    m = re.search(r"(高于|超过|大于|多于)\s*(\d+(?:\.\d+)?)\s*%?", text)
    if m:
        return {"field": "value", "op": "gt", "value": float(m.group(2)) / 100.0, "color": "red"}
    return None
```

```
__all__ = ["ParseResult", "parse_nl_to_schema"]
# ----- FILE: packages/chiyu-portal/src/chiyu_portal/onboarding.py -----
"""引导式页面导航（对标 DeepSeek Harness 桌面版的 onboarding 向导）。

深源：登录 → 选工作区 → 选方向 → 选工作流 → 选工作模式。
本仓落点：chiyu-portal 的引导式向导，菜单与可选工作流**全部**从
`chiyu_kernel.plugins` 的 `PluginRegistry` / `MODE_PRESETS` 动态生成，门户零领域字面量。

⛔ 方向（行业/租户）经 `TenantProfile`（L4）注入，本模块不写任何领域词。
⛔ 只从 `chiyu_kernel.api` 门面取插件能力，不 import 内核内部路径。
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, List, Tuple

from chiyu_kernel.api import MODE_PRESETS, PluginRegistry, register_builtins


def onboarding_steps() -> List[Dict[str, str]]:
    """四步向导（第 3 步高亮 = 当前选择工作模式）。"""
    return [
        {"n": "1", "key": "direction", "label": "选方向",
         "hint": "行业 / 租户（经 TenantProfile 注入）"},
        {"n": "2", "key": "workflow", "label": "选工作流",
         "hint": "从已注册能力动态生成"},
        {"n": "3", "key": "mode", "label": "选工作模式", "hint": "标准 / PTC / 极简 / 创造",
         "active": "true"},
        {"n": "4", "key": "workspace", "label": "进入工作台", "hint": "按模式装配能力"},
    ]


def mode_cards() -> List[Dict[str, object]]:
    """从 WorkMode 预设动态生成模式卡片（含启用插件、Phase B 后端）。"""
    reg = PluginRegistry()
    register_builtins(reg)
    cards: List[Dict[str, object]] = []
    for mid in sorted(MODE_PRESETS):
        mode = MODE_PRESETS[mid]
        enabled = [p for p in mode.plugins if reg.has(p)]
        cards.append({
            "id": mode.id,
            "label": mode.label,
            "description": mode.description,
            "plugins": enabled,
            "phase_b_backend": mode.phase_b_backend,
            "default": mode.default,
        })
```

```
    return cards


def level_cards() -> List[Dict[str, object]]:
    """场景三级分类卡片（解法分层，中性通用枚举）。

    边界判据：AI 替几个岗位、替几次 —— 而非系统数量 / 链路长短。
    ⛔ 零领域字面量：真实场景画像由消费方经 TenantProfile 覆盖注入。
    `recommended_modes` 为层级 → 工作模式联动推荐（向导第 3 步高亮依据）。
    """
    return [
        {
            "id": "single",
            "label": "单点提效",
            "criteria": "单智能体取数算出完整结果，交回一个岗位一次确认；人只提供输入 + 一次审批",
            "solution": "沉淀为可复用 SKILL / 单 Agent，低成本、快见效、易治理",
            "recommended_modes": ["ptc", "minimal"],
        },
        {
            "id": "flow",
            "label": "流程提效",
            "criteria": "AI 输出喂送下一个岗位做不同的事，多岗位接力；每环节有不同角色的独立动作 / 批准",
            "solution": "智能体编排：角色智能体 + 状态机 + 事件总线 + 审批闸门（HITL）",
            "recommended_modes": ["standard"],
        },
        {
            "id": "end2end",
            "label": "端到端增值",
            "criteria": "从开始到结束改变经营结果：感知 → 认知 → 决策 → 执行闭环，AI 完成决策、人做审核",
            "solution": "企业级 AI 决策平台：统一决策内核 + 记忆系统 + 治理（RBAC / 审计）",
            "recommended_modes": ["standard", "creative"],
        },
    ]


def workflow_cards() -> List[Dict[str, object]]:
    """工作流卡片（中性通用枚举，`level` = 归属三级分类，向导第 2 步分组依据）。"""
    return [
        {"id": "decide", "label": "决策辅助",
         "hint": "决策脊柱 • 红线校验 • 闭环审核", "level": "end2end"},
        {"id": "research", "label": "调研检索",
         "hint": "联网 + 子智能体归纳，一次交回", "level": "single"},
        {"id": "automate", "label": "任务编排",
         "hint": "跨岗位接力 • 状态机 + 审批闸门", "level": "flow"},
        {"id": "build", "label": "能力沉淀",
         "hint": "封装为 SKILL / 插件，单点复用", "level": "single"},
    ]


def direction_cards() -> List[Dict[str, str]]:
```

```
    """方向卡片（行业 / 租户演示枚举）。

    ⛔ 真实方向清单经 TenantProfile（L4）注入后由此覆盖；本枚举仅演示兜底。
    """
    return [
        {"id": "finance", "label": "金融 / 风控", "hint": "信贷、反欺诈、合规"},
        {"id": "retail", "label": "零售 / 电商", "hint": "选品、运营、客服"},
        {"id": "manufacturing", "label": "制造 / 供应链", "hint": "排程、质量、履约"},
        {"id": "generic", "label": "通用助手", "hint": "跨领域问答与任务"},
    ]


def render_onboarding() -> "OnboardingResult":
    """渲染引导式导航 HTML（自主产出，不经 NL→Schema 链路，属静态向导页）。"""
    html = _build_html(onboarding_steps(), mode_cards())
    return OnboardingResult(ok=True, html=html, errors=())


@dataclass
class OnboardingResult:
    ok: bool
    html: str
    errors: Tuple[str, ...]


# ——— HTML 生成（轻量、自包含）——————————————————————————————————————————————————————————
# ⛔ HTML 属性统一用单引号，CSS 块不走 f-string（避免 CSS 花括号被当成表达式）。
_CSS = """
<style>
 body{font-family:-apple-system,Segoe UI,Roboto,'Microsoft YaHei',sans-serif;
   margin:0;background:#f8fafc;color:#0f172a;}
 .ob-wrap{max-width:920px;margin:0 auto;padding:28px 20px;}
 h1{font-size:20px;margin:0 0 4px;}
 .ob-sub{color:#64748b;font-size:13px;margin:0 0 18px;}
 .ob-steps{display:flex;gap:8px;flex-wrap:wrap;margin-bottom:22px;}
 .ob-step{padding:7px 12px;border-radius:20px;background:#e0f2fe;color:#075985;
   border:1px solid #7dd3fc;font-size:13px;font-weight:600;}
 .ob-step.ob-active{background:#16a34a;color:#fff;border-color:#16a34a;
   box-shadow:0 0 0 3px rgba(22,163,74,.15);}
 .ob-cards{display:grid;grid-template-columns:repeat(2,1fr);gap:14px;}
 .ob-card{background:#fff;border:1px solid #e2e8f0;border-radius:12px;padding:14px;}
 .ob-card.ob-default{border-color:#16a34a;}
 .ob-card h3{margin:0 0 6px;font-size:15px;}
 .ob-tag{font-size:10px;font-weight:600;color:#16a34a;
   border:1px solid #16a34a;border-radius:8px;padding:1px 6px;vertical-align:middle;}
 .ob-card p{margin:0 0 8px;font-size:12.5px;color:#475569;line-height:1.6;}
 .ob-plugins{display:flex;gap:5px;flex-wrap:wrap;margin-bottom:8px;}
 .ob-pill{font-size:11px;padding:2px 8px;border-radius:10px;background:#dcfce7;color:#166534;}
 .ob-pb{font-size:11px;color:#0891b2;}
</style>"""
```

```
def _build_html(steps: List[Dict[str, str]],
                cards: List[Dict[str, object]]) -> str:
    step_parts: List[str] = []
    for s in steps:
        active = " ob-active" if s.get("active") else ""
        step_parts.append(
            f"<span class='ob-step{active}'><b>{s['n']}</b> {s['label']}</span>")
    step_html = "".join(step_parts)

    card_parts: List[str] = []
    for c in cards:
        cls = "ob-card ob-default" if c["default"] else "ob-card"
        tag = " <span class='ob-tag'>DEFAULT</span>" if c["default"] else ""
        pills = "".join(f"<span class='ob-pill'>{p}</span>" for p in c["plugins"])
        if c["phase_b_backend"]:
            pb = f"<div class='ob-pb'>Phase B → {c['phase_b_backend']}</div>"
        else:
            pb = "<div class='ob-pb'>无需 LLM（确定性）</div>"
        card_parts.append(
            f"<div class='{cls}'><h3>{c['label']}{tag}</h3>"
            f"<p>{c['description']}</p>"
            f"<div class='ob-plugins'>{pills}</div>{pb}</div>")
    card_html = "".join(card_parts)

    return (
        "<!DOCTYPE html><html lang='zh-CN'><head><meta charset='utf-8'>"
        "<title>ChiYu 引导式导航 • 选工作模式</title>" + _CSS +
        "</head><body><div class='ob-wrap'>"
        "<h1>ChiYu 引导式页面导航</h1>"
        "<p class='ob-sub'>选方向 → 选工作流 → 选工作模式 → 进入工作台。菜单与可选工作流"
        "均从插件注册表动态生成；方向经 TenantProfile 注入，门户零领域字面量。</p>"
        f"<div class='ob-steps'>{step_html}</div>"
        f"<div class='ob-cards'>{card_html}</div>"
        "</div></body></html>"
    )


__all__ = [
    "onboarding_steps", "mode_cards", "level_cards", "workflow_cards",
    "direction_cards", "render_onboarding", "OnboardingResult",
]
# ----- FILE: packages/chiyu-portal/src/chiyu_portal/page_store.py -----
"""页面生命周期：护栏 → 解析 → 校验 → 落库 draft → 发布。

无 PG 可全绿（内存存储，便于测试；消费方可替换为真实存储）。
三段式全链路在此编排：``guard_nl_input`` → ``parse_nl_to_schema`` → ``validate_page_schema``。
"""
from __future__ import annotations
```

```
import uuid
from dataclasses import dataclass, field
from typing import Any

from .guardrails import GuardResult, guard_nl_input
from .nl_parser import ParseResult, parse_nl_to_schema
from .renderer import RenderResult, render_page
from .schema import ValidationError, validate_page_schema


@dataclass
class PageDraft:
    """页面草稿（未发布）。"""

    draft_id: str
    schema: dict[str, Any]
    confidence: float
    needs_clarification: bool
    notes: list[str] = field(default_factory=list)
    published: bool = False


@dataclass
class CreateResult:
    """创建结果。``ok=False`` 时携带护栏原因或字段级校验错误。"""

    ok: bool
    draft: PageDraft | None = None
    errors: tuple[ValidationError, ...] = ()
    guard_reason: str = ""

    def __bool__(self) -> bool:
        return self.ok


class PageStore:
    """内存页面存储（无 PG 依赖）。消费方应替换为真实持久层。"""

    def __init__(self) -> None:
        self._drafts: dict[str, PageDraft] = {}

    def create_page_from_nl(
        self,
        text: str,
        *,
        particle_types: Any,
        entity_map: dict[str, str] | None = None,
        allowed_actions=None,
        status_fields: Any = (),
```

```
        snapshot_fields: Any = (),
    ) -> CreateResult:
        guard: GuardResult = guard_nl_input(text)
        if not guard:
            return CreateResult(False, guard_reason=guard.reason)

        parsed: ParseResult = parse_nl_to_schema(text, entity_map=entity_map)
        # 实体未识别（低置信 / needsClarification）→ 无粒子绑定，不应落草稿
        if parsed.needs_clarification:
            return CreateResult(
                False, errors=(ValidationError("nl", "实体未识别，需澄清后再生成页面"),),
            )
        errors = validate_page_schema(
            parsed.schema, particle_types=particle_types,
            allowed_actions=allowed_actions, status_fields=status_fields,
            snapshot_fields=snapshot_fields,
        )
        if errors:
            return CreateResult(False, errors=errors)

        draft = PageDraft(
            draft_id=uuid.uuid4().hex,
            schema=parsed.schema,
            confidence=parsed.confidence,
            needs_clarification=parsed.needs_clarification,
            notes=parsed.notes,
        )
        self._drafts[draft.draft_id] = draft
        return CreateResult(True, draft=draft)

    def publish_page(self, draft_id: str) -> bool:
        draft = self._drafts.get(draft_id)
        if draft is None:
            return False
        draft.published = True
        return True

    def render(self, draft_id: str, **kwargs: Any) -> RenderResult:
        draft = self._drafts.get(draft_id)
        if draft is None:
            return RenderResult(False, errors=(ValidationError("draft", "草稿不存在"),))
        return render_page(draft.schema, **kwargs)


__all__ = ["PageDraft", "CreateResult", "PageStore"]
# ----- FILE: packages/chiyu-portal/src/chiyu_portal/renderer.py -----
"""渲染层：受控 Schema → HTML。⛔ 唯一渲染出口；渲染前再校验；动态值全转义；
交互仅 data-action 声明式；输出强制无 <script> / on*=。纵深防御第三层。

⛔ 判别力：非法 Schema 直接拒绝（返回字段级错误），绝不尽力渲染。
```

```
"""
from __future__ import annotations

import html
import re
from dataclasses import dataclass
from typing import Any, Iterable

from .schema import SCHEMA_VERSION, ValidationError, validate_page_schema

_SCRIPT_RE = re.compile(r"<\s*script", re.IGNORECASE)
_ONATTR_RE = re.compile(r"\son\w+\s*=", re.IGNORECASE)


@dataclass
class RenderResult:
    """渲染结果。``ok=False`` 时携带字段级错误（非法 Schema 被拒）。"""

    ok: bool
    html: str = ""
    errors: tuple[ValidationError, ...] = ()

    def __bool__(self) -> bool:
        return self.ok


def render_page(
    schema: dict[str, Any],
    *,
    particle_types: Any,
    allowed_actions: Iterable[str] | None = None,
    status_fields: Iterable[str] = (),
    snapshot_fields: Iterable[str] = (),
) -> RenderResult:
    """渲染受控 Schema 为 HTML。非法 Schema 拒绝（字段级错误），绝不尽力渲染。"""
    errors = validate_page_schema(
        schema, particle_types=particle_types,
        allowed_actions=allowed_actions, status_fields=status_fields,
        snapshot_fields=snapshot_fields,
    )
    if errors:
        return RenderResult(False, errors=errors)

    out: list[str] = []
    out.append("<!DOCTYPE html>")
    out.append('<html lang="zh"><head><meta charset="utf-8">')
    out.append("<title>" + html.escape(str(schema.get("title", ""))) + "</title>")
    out.append('<link rel="stylesheet" href="/portal/page.css">')
    out.append("</head><body>")
    out.append('<main class="portal-page">')
```

```
    out.append('<h1 class="page-title">' + html.escape(str(schema.get("title", ""))) + "</h1>")
    for comp in schema.get("components", []) or []:
        out.append(_render_component(comp))
    out.append("</main></body></html>")

    rendered = "".join(out)
    # 渲染层硬闸门：不得出现 script / 内联事件属性
    if _SCRIPT_RE.search(rendered) or _ONATTR_RE.search(rendered):
        return RenderResult(False, errors=(ValidationError("render", "渲染产物含禁用标记"),))
    return RenderResult(True, html=rendered)


def _render_component(comp: dict[str, Any]) -> str:
    kind = html.escape(str(comp.get("kind", "")))
    title = html.escape(str(comp.get("title", "")))
    parts: list[str] = [f'<section class="component component-{kind}">']
    parts.append(f'<h2 class="component-title">{title}</h2>')
    binding = comp.get("dataBinding")
    if isinstance(binding, dict):
        pt = html.escape(str(binding.get("particleType", "")))
        parts.append(f'<div class="binding" data-particle-type="{pt}">')
        for m in binding.get("metrics", []) or []:
            if not isinstance(m, dict):
                continue
            label = html.escape(str(m.get("label", "")))
            field = html.escape(str(m.get("field", "")))
            parts.append(f'<span class="metric" data-field="{field}">{label}</span>')
        parts.append("</div>")
    for act in comp.get("actions", []) or []:
        if not isinstance(act, dict):
            continue
        label = html.escape(str(act.get("label", "")))
        action = html.escape(str(act.get("action", "")))
        # 交互仅声明式 data-action，无 onclick / 内联 JS
        parts.append(f'<button class="action-btn" data-action="{action}">{label}</button>')
    parts.append("</section>")
    return "".join(parts)


def render_conversation(
    *,
    events: Any = None,
    composer_action: str = "send_message",
    goal: str = "",
) -> "RenderResult":
    """渲染 dsh 式对话页：session 日志 + 目标栏 + composer。

    声明式 ``data-action``，无内联 JS；复用渲染层硬闸门（无 <script> / on*=）。
    ``events`` 形如 ``[{"role": "user"|"assistant", "content": ...}]`` 或
    ``[{"kind": "user/message"|"assistant/message", "payload": {"content": ...}}]``
```

```
    （兼容 SessionEvent 形态，零领域字面量）。
    """
    out: list[str] = []
    out.append("<!DOCTYPE html>")
    out.append('<html lang="zh"><head><meta charset="utf-8">'
               '<title>对话</title><link rel="stylesheet" href="/portal/page.css"></head><body>')
    out.append('<main class="portal-page conversation-page">')
    if goal:
        out.append(f'<div class="goal-bar">{html.escape(goal)}</div>')
    out.append('<section class="component component-chat-log">'
               '<h2 class="component-title">对话</h2><div class="chat-log">')
    for ev in events or []:
        role = html.escape(str(ev.get("role", ev.get("kind", ""))))
        payload = ev.get("payload") or {}
        content = html.escape(str(ev.get("content", payload.get("content", ""))))
        out.append(f'<div class="msg msg-{role}">'
                   f'<span class="role">{role}</span>'
                   f'<span class="text">{content}</span></div>')
    out.append('</div></section>')
    out.append(f'<form class="composer" data-action="{html.escape(composer_action)}">'
               '<input name="message" class="composer-input" placeholder="输入消息…" '
               'autocomplete="off">'
               '<button type="submit" class="action-btn">发送</button></form>')
    out.append('</main></body></html>')

    rendered = "".join(out)
    if _SCRIPT_RE.search(rendered) or _ONATTR_RE.search(rendered):
        return RenderResult(False, errors=(ValidationError("render", "渲染产物含禁用标记"),))
    return RenderResult(True, html=rendered)


__all__ = ["RenderResult", "render_page", "render_conversation"]
# ----- FILE: packages/chiyu-portal/src/chiyu_portal/schema.py -----
"""chiyu-portal 受控页面 Schema 协议（零领域字面量）。

三段式：NL → 受控 Schema → 运行时渲染器。本模块是 **Schema 层**（结构化中间表示），
定义协议常量与校验器。渲染器（`renderer.py`）是唯一渲染出口，且渲染前会再次调用
`validate_page_schema` —— 非法 Schema 必须被拒（不是「尽力渲染」）。

⛔ 粒子类型 / 动作经**注入**，框架层零硬编码业务字面量。
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Iterable

from chiyu_kernel.contract.particle import ParticleTypeSet

# ─── 协议常量（形态，无业务取值）──────────────────────────────────────────────
PAGE_TYPES: tuple[str, ...] = (
```

```
    "dashboard", "table", "form", "detail", "workspace", "approval",
    "conversation",
)
COMPONENT_KINDS: tuple[str, ...] = (
    "metric-card", "table", "form", "detail",
    "reasoning-trace", "approval-queue", "conflict-marker",
    "goal-form", "interaction-panel", "acceptance-checklist",
    "chat-log", "composer", "goal-bar",
)
THEMES: tuple[str, ...] = ("light", "dark")
FILTER_OPS: tuple[str, ...] = ("eq", "ne", "lt", "gt", "lte", "gte", "in")
AGG_FUNCS: tuple[str, ...] = ("count", "sum", "avg", "min", "max", "latest")

# 框架级默认动作白名单（消费方注入其完整 ActionRegistry 覆盖）
DEFAULT_ALLOWED_ACTIONS: tuple[str, ...] = (
    "data-particle-read", "data-particle-write", "page-reload", "aiattr-generate",
)

SCHEMA_VERSION = "0.2"


@dataclass(frozen=True)
class ValidationError:
    """字段级校验错误（路径定位，便于前端逐字段回显）。"""

    path: str
    message: str

    def __str__(self) -> str:
        return f"{self.path}: {self.message}"


def validate_page_schema(
    schema: dict[str, Any],
    *,
    particle_types: ParticleTypeSet,
    allowed_actions: Iterable[str] | None = None,
    status_fields: Iterable[str] = (),
    snapshot_fields: Iterable[str] = (),
) -> tuple[ValidationError, ...]:
    """校验受控页面 Schema。返回字段级错误元组（空 = 通过）。

    ⛔ 判别力自证要点（均须被拒，非静默映射）：
      - 未知 particleType / 页面类型 / 组件类型
      - 非白名单 action
      - 非法过滤算子 / 聚合函数
      - 状态字段非 eq 或聚合
      - 快照字段非 latest
    """
    if not isinstance(schema, dict):
```

```
        return (ValidationError("schema", "Schema 必须是对象"),)

    errors: list[ValidationError] = []

    version = schema.get("version")
    if version is not None and version != SCHEMA_VERSION:
        errors.append(ValidationError("version", f"不支持的 version: {version!r}"))

    ptype = schema.get("type")
    if ptype is not None and ptype not in PAGE_TYPES:
        errors.append(ValidationError("type", f"未知页面类型: {ptype!r}"))

    layout = schema.get("layout")
    if isinstance(layout, dict):
        theme = layout.get("theme")
        if theme is not None and theme not in THEMES:
            errors.append(ValidationError("layout.theme", f"未知主题: {theme!r}"))

    actions = set(allowed_actions) if allowed_actions is not None else set(DEFAULT_ALLOWED_ACTIONS)
    status_set = set(status_fields)
    snapshot_set = set(snapshot_fields)

    for i, comp in enumerate(_iter_components(schema)):
        cpath = f"components[{i}]"
        kind = comp.get("kind")
        if kind is not None and kind not in COMPONENT_KINDS:
            errors.append(ValidationError(f"{cpath}.kind", f"未知组件类型: {kind!r}"))

        binding = comp.get("dataBinding")
        if isinstance(binding, dict):
            _validate_binding(
                binding, f"{cpath}.dataBinding", errors,
                particle_types, status_set, snapshot_set,
            )

        for j, act in enumerate(comp.get("actions", []) or []):
            apath = f"{cpath}.actions[{j}]"
            aname = act.get("action") if isinstance(act, dict) else None
            if aname is not None and aname not in actions:
                errors.append(ValidationError(f"{apath}.action", f"非白名单动作: {aname!r}"))

    return tuple(errors)


def _iter_components(schema: dict[str, Any]) -> list[dict]:
    comps = schema.get("components")
    return comps if isinstance(comps, list) else []


def _validate_binding(
```

```
    binding: dict[str, Any], path: str, errors: list[ValidationError],
    particle_types: ParticleTypeSet, status_set: set[str], snapshot_set: set[str],
) -> None:
    src = binding.get("source")
    if src is not None and src != "particle":
        errors.append(ValidationError(f"{path}.source", f"仅支持 particle 数据源: {src!r}"))

    pt = binding.get("particleType")
    if pt is not None and pt not in particle_types.names:
        errors.append(ValidationError(f"{path}.particleType", f"未知粒子类型: {pt!r}（受控枚举）"))

    for fi, f in enumerate(binding.get("filters", []) or []):
        if not isinstance(f, dict):
            continue
        fpath = f"{path}.filters[{fi}]"
        op = f.get("op")
        field_name = f.get("field")
        if op is not None and op not in FILTER_OPS:
            errors.append(ValidationError(f"{fpath}.op", f"非法过滤算子: {op!r}"))
        if field_name in status_set and op is not None and op != "eq":
            errors.append(ValidationError(f"{fpath}.op", f"状态字段仅支持 eq: {field_name!r}"))

    for mi, m in enumerate(binding.get("metrics", []) or []):
        if not isinstance(m, dict):
            continue
        mpath = f"{path}.metrics[{mi}]"
        agg = m.get("agg")
        field_name = m.get("field")
        if agg is not None and agg not in AGG_FUNCS:
            errors.append(ValidationError(f"{mpath}.agg", f"非法聚合函数: {agg!r}"))
        if field_name in status_set and agg is not None:
            errors.append(ValidationError(f"{mpath}.agg", f"状态字段不可聚合: {field_name!r}"))
        if field_name in snapshot_set and agg is not None and agg != "latest":
            errors.append(ValidationError(f"{mpath}.agg", f"快照字段仅支持 latest: {field_name!r}"))


__all__ = [
    "PAGE_TYPES", "COMPONENT_KINDS", "THEMES", "FILTER_OPS", "AGG_FUNCS",
    "DEFAULT_ALLOWED_ACTIONS", "SCHEMA_VERSION", "ValidationError", "validate_page_schema",
]

# ======================================================================
# PACKAGE: chiyu-cli  —— 命令行入口
# ======================================================================
# ----- FILE: packages/chiyu-cli/pyproject.toml -----
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
```

```
name = "chiyu-cli"
version = "0.1.0"
description = "ChiYu AI 原生决策平台终端运行入口（CLI + REPL）"
requires-python = ">=3.11"

[project.scripts]
chiyu = "chiyu_cli.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src", "../chiyu-kernel/src", "../chiyu-decision/src", "../chiyu-portal/src"]
testpaths = ["tests"]
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/__init__.py -----
"""chiyu-cli：ChiYu AI 原生决策平台终端运行入口（CLI + 受限命令壳 / REPL）。

⛔ 分层纪律：本包只消费 `chiyu_kernel.api` / `chiyu_kernel.contract.*`
   与兄弟包的 `*.api` 门面，**禁止** import 内部子模块。
"""
from __future__ import annotations

__version__ = "0.1.0"

__all__ = ["__version__"]
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/__main__.py -----
"""支持 `python -m chiyu_cli`（零安装运行）。"""
from __future__ import annotations

import sys

from .main import main

if __name__ == "__main__":
    sys.exit(main())
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/__init__.py -----
"""chiyu 子命令实现。

每个子模块对外只暴露 `register(subparsers)` 与 `handler(args)` 两个契约函数。
"""
from __future__ import annotations

__all__ = ["decide", "dev", "evolution", "portal", "selftest"]
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/_plugins.py -----
"""CLI 侧的插件/模式辅助（薄封装，避免各命令重复装配 registry）。

⛔ 只从 `chiyu_kernel.api`/公开门面取插件能力，不 import 内核内部路径。
"""
from __future__ import annotations
```

```
# ⛔ 走门面而非内部路径（`chiyu_kernel.plugins`）：sanctioned import 面见
#    docs/sanctioned-import-surface.md，由 tools/check_import_surface.py 机械校验。
from chiyu_kernel.api import (
    MODE_PRESETS,
    PluginRegistry,
    register_builtins,
    resolve_work_mode,
)


def build_registry() -> PluginRegistry:
    """装配含 6 个内置插件的 registry。"""
    reg = PluginRegistry()
    register_builtins(reg)
    return reg


def resolve(mode_id: str | None):
    """解析工作模式，返回 (WorkMode, enabled, disabled, warnings)。永不抛。"""
    return resolve_work_mode(mode_id, build_registry())


def mode_summary_lines() -> list[str]:
    """给 `chiyu modes` 用的多行摘要。"""
    reg = build_registry()
    lines: list[str] = []
    for mid in sorted(MODE_PRESETS):
        mode = MODE_PRESETS[mid]
        _, enabled, disabled, _ = resolve(mid)
        flag = "（默认）" if mode.default else ""
        lines.append(f"● {mode.id}{flag} —— {mode.label}")
        lines.append(f"    {mode.description}")
        lines.append(f"    启用插件：{', '.join(enabled) or '（无）'}")
        if disabled:
            lines.append(f"    惰性关闭：{', '.join(disabled)}")
        if mode.phase_b_backend:
            lines.append(f"    Phase B 后端：{mode.phase_b_backend}")
    return lines
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/decide.py -----
"""chiyu decide —— 跑决策脊柱（确定性分支，零 IO、不接 LLM）。

⛔ 诚实原则：八要素未提供的项一律留空，由脊柱如实报 missing；
   绝不伪造齐备以换取「看起来跑通」。
"""
from __future__ import annotations

import argparse
import json
from typing import Any, Dict
```

```
from chiyu_decision.api import run_spine
from chiyu_kernel.contract.decision import RUBRIC_KEYS
from ._plugins import resolve

DEFAULT_SCORE = 3          # RUBRIC_SCALE_MAX=4；全 3 ⇒ ratio=27/36=0.75 < 0.79 阈值
DEFAULT_THRESHOLD = 0.79


def _parse_scores(raw: str | None) -> Dict[str, float]:
    """解析 `--scores clarity=3,logic=4`；未提及的键取 DEFAULT_SCORE。"""
    scores: Dict[str, float] = {k: float(DEFAULT_SCORE) for k in RUBRIC_KEYS}
    if not raw:
        return scores
    for item in raw.split(","):
        item = item.strip()
        if not item:
            continue
        if "=" not in item:
            raise ValueError(f"评分项须形如 key=value：{item!r}")
        key, _, value = item.partition("=")
        key = key.strip()
        if key not in RUBRIC_KEYS:
            raise ValueError(f"未知尺子键 {key!r}；合法键：{', '.join(RUBRIC_KEYS)}")
        scores[key] = float(value.strip())
    return scores


def _build_elements(topic: str) -> Dict[str, object]:
    """八要素：只填真实提供的项（intent = topic）。

    ⛔ 不得用空串/占位补齐其余键 —— `check_elements` 只判**键是否存在**，
       补空键会让「要素缺失」恒为「无」，属典型假齐备。CLI 只拿到了议题，
       故如实只给 intent，其余由脊柱报 missing。
    """
    return {"intent": topic}


def handler(args: argparse.Namespace) -> int:
    scores = _parse_scores(args.scores)
    elements = _build_elements(args.topic)
    mode, enabled, disabled, mwarn = resolve(args.mode)
    result = run_spine(
        context={
            "decision_id": args.decision_id or f"cli-{args.topic[:24]}",
            "topic": args.topic,
            "work_mode": mode.id,        # 观测面：本次走的工作模式
        },
        profile=None,                    # ⛔ 无默认值，预览模式显式传 None
        rubric_scores=scores,
        elements=elements,
```

```
        persister=None,                # 不落库
        threshold=args.threshold,
    )
    payload: Dict[str, Any] = {
        "decision_id": result.decision_id,
        "stage_reached": result.stage_reached,
        "confidence": round(result.confidence, 4),
        "autonomous": result.autonomous,
        "redline_hit": result.redline_hit,
        "redline_reasons": list(result.redline_reasons),
        "rubric_missing": list(result.rubric_missing),
        "elements_missing": list(result.elements_missing),
        "persisted": result.persisted,
        "work_mode": mode.id,
        "work_mode_plugins": list(enabled),
    }
    if args.json:
        print(json.dumps(payload, ensure_ascii=False, indent=2))
        return 0
    print(f"decision_id   : {payload['decision_id']}")
    print(f"stage_reached : {payload['stage_reached']}")
    print(f"confidence    : {payload['confidence']}  (阈值 {args.threshold})")
    print(f"autonomous    : {payload['autonomous']}")
    print(f"redline_hit   : {payload['redline_hit']} {payload['redline_reasons'] or ''}")
    print(f"尺子缺失      : {', '.join(payload['rubric_missing']) or '无'}")
    print(f"要素缺失      : {', '.join(payload['elements_missing']) or '无'}")
    print(f"工作模式      : {payload['work_mode']}（启用插件：{', '.join(payload['work_mode_plugins']) or '无'}）")
    return 0


def register(sub: Any) -> None:
    p = sub.add_parser("decide", help="跑决策脊柱（确定性分支）")
    p.add_argument("--topic", required=True, help="决策议题描述")
    p.add_argument("--scores", default=None, help="九尺子评分，如 clarity=4,logic=3")
    p.add_argument("--threshold", type=float, default=DEFAULT_THRESHOLD, help="自主放行阈值")
    p.add_argument("--decision-id", default=None, help="指定决策 ID")
    p.add_argument("--mode", default=None,
                   help="工作模式：standard/ptc/minimal/creative（缺省=standard）")
    p.add_argument("--json", action="store_true", help="机器可读输出")
    p.set_defaults(handler=handler)
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/dev.py -----
"""chiyu dev —— 完整 Python REPL（⚠ 仅 full 全代码版；external 构建不打包本模块）。

预置三门面与常用快捷函数，供内部开发现场试跑。
"""
from __future__ import annotations

import argparse
import code
from pathlib import Path
```

```
from typing import Any, Dict

BANNER = """chiyu dev —— 完整 Python REPL（仅全代码版）
预置：K = chiyu_kernel.api   C = chiyu_kernel.contract
      D = chiyu_decision.api P = chiyu_portal.api
      render(nl, out=None) • readiness() • decide(topic)
Ctrl-D 退出。"""


def _namespace() -> Dict[str, Any]:
    import chiyu_kernel.api as K
    import chiyu_kernel.contract as C
    import chiyu_decision.api as D
    import chiyu_portal.api as P
    from chiyu_kernel.contract.particle import CORE_ROLES, ParticleTypeSet

    def render(nl: str, out: str | None = None) -> str:
        guard = P.guard_nl_input(nl)
        if not guard.ok:
            return f"护栏拒绝：{guard.reason}"
        parsed = P.parse_nl_to_schema(nl)
        res = P.render_page(parsed.schema, particle_types=ParticleTypeSet(names=CORE_ROLES))
        if not res.ok:
            return "渲染拒绝：" + "; ".join(str(e) for e in res.errors)
        if out:
            Path(out).write_text(res.html, encoding="utf-8")
            return f"已写出 {out}"
        return res.html

    def readiness() -> str:
        return K.readiness_report().summary

    def decide(topic: str) -> Any:
        return D.run_spine(
            context={"topic": topic}, profile=None,
            rubric_scores={}, elements={"intent": topic}, persister=None,
        )

    return {
        "K": K, "C": C, "D": D, "P": P,
        "render": render, "readiness": readiness, "decide": decide,
    }


def handler(args: argparse.Namespace) -> int:
    code.interact(banner=BANNER, local=_namespace())
    return 0


def register(sub: Any) -> None:
```

```
    p = sub.add_parser("dev", help="完整 Python REPL（仅全代码版）")
    p.set_defaults(handler=handler)
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/evolution.py -----
"""chiyu evolution readiness —— 自主通道开闸前置认证（§12.3 七闸 + §12.4 G1/G2/G3）。

⛔ 本命令**只认证、不开闸**；夜批调度器上线前必须先过本闸。
"""

from __future__ import annotations

import argparse
import json
import sys
from typing import Any

from chiyu_kernel.api import (
    ChannelNotReadyError,
    assert_channel_may_open,
    readiness_report,
)


def handler(args: argparse.Namespace) -> int:
    if args.assert_:
        try:
            report = assert_channel_may_open()
        except ChannelNotReadyError as exc:
            rep = exc.args[0]
            print("[拒绝开闸] 存在阻断闸 RED：", file=sys.stderr)
            for c in rep.blocking_red:
                print(f"  - {c.check_id} {c.title}: {c.detail}", file=sys.stderr)
            return 1
    else:
        report = readiness_report()

    if args.json:
        rows = [
            {
                "check_id": c.check_id,
                "title": c.title,
                "status": c.status,
                "detail": c.detail,
                "discriminating": c.discriminating,
            }
            for c in report.checks
        ]
        print(json.dumps(rows, ensure_ascii=False, indent=2))
        return 0

    print(f"{'ID':<28}{'状态':<8}{'有鉴别力':<10}标题")
    print("-" * 76)
```

```
    for c in report.checks:
        flag = "是" if c.discriminating else "否"
        print(f"{c.check_id:<28}{c.status:<8}{flag:<10}{c.title}")
        if c.detail:
            print(f"{'':<28}{c.detail}")
    print("-" * 76)
    print(report.summary())          # ⚠ summary 是方法，不是属性
    return 0


def register(sub: Any) -> None:
    p = sub.add_parser("evolution", help="平台自进化")
    sub2 = p.add_subparsers(dest="evolution_cmd", required=True)
    r = sub2.add_parser("readiness", help="开闸前置认证")
    r.add_argument("--assert", dest="assert_", action="store_true",
                   help="未全绿即抛 ChannelNotReadyError 并退 1")
    r.add_argument("--json", action="store_true", help="机器可读输出")
    r.set_defaults(handler=handler)
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/modes.py -----
"""chiyu modes —— 列出四种工作模式及其启用插件（从 PluginRegistry 动态生成）。

菜单内容完全来自 `chiyu_kernel.plugins` 的预设与 registry，不硬编码领域词。
"""
from __future__ import annotations

import argparse
import sys
from typing import Any

from ._plugins import mode_summary_lines, resolve


def handler(args: argparse.Namespace) -> int:
    if args.mode:
        mode, enabled, disabled, warns = resolve(args.mode)
        if warns:
            for w in warns:
                print(f"[警告] {w}", file=sys.stderr)
        print(f"工作模式：{mode.id} —— {mode.label}")
        print(f"说明    ：{mode.description}")
        print(f"启用插件：{', '.join(enabled) or '（无）'}")
        if disabled:
            print(f"惰性关闭：{', '.join(disabled)}")
        print(f"Phase A 后端：{mode.llm_backend}"
              + (f"（Phase B → {mode.phase_b_backend}）" if mode.phase_b_backend else "（无需 LLM）"))
        return 0
    print("ChiYu 工作模式（对标 DeepSeek Harness 桌面版）：")
    print("=" * 56)
    for line in mode_summary_lines():
        print(line)
```

```
    return 0


def register(sub: Any) -> None:
    p = sub.add_parser("modes", help="列出四种工作模式及启用插件")
    p.add_argument("--mode", default=None,
                   help="查看单个模式详情：standard/ptc/minimal/creative")
    p.set_defaults(handler=handler)
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/portal.py -----
"""chiyu portal render —— NL → 受控 Schema → HTML（零 Web 服务，落盘即可看）。

链路：guard_nl_input（注入护栏）→ parse_nl_to_schema（绝不直出 HTML）
     → render_page（唯一渲染出口，非法 Schema 字段级拒绝）。
"""
from __future__ import annotations

import argparse
import sys
from pathlib import Path
from typing import Any, Dict, List

from chiyu_kernel.contract.particle import CORE_ROLES, ParticleTypeSet
from chiyu_portal.api import guard_nl_input, parse_nl_to_schema, render_page, render_onboarding
from ._plugins import resolve


def _parse_entity_map(raw: str | None) -> Dict[str, str]:
    """解析 `--entity-map 客户=entity,订单=entity`（NL 实体词 → 粒子类型）。

    ⛔ 映射值由**调用方注入** —— 内核零领域字面量，CLI 不得内置任何业务实体词。
    """
    mapping: Dict[str, str] = {}
    if not raw:
        return mapping
    for item in raw.split(","):
        item = item.strip()
        if not item:
            continue
        if "=" not in item:
            raise ValueError(f"实体映射项须形如 实体词=粒子类型：{item!r}")
        token, _, ptype = item.partition("=")
        mapping[token.strip()] = ptype.strip()
    return mapping


def _particle_set(raw: str | None, entity_map: Dict[str, str]) -> ParticleTypeSet:
    """默认核心角色 + `--particle-types` + 实体映射值（后者否则 Schema 校验会拒）。"""
    extra: List[str] = []
    if raw:
        extra += [x.strip() for x in raw.split(",") if x.strip()]
```

```
    extra += [p for p in entity_map.values() if p]
    # ⚠ 必须对 CORE_ROLES + extra 整体去重：映射值可能与核心角色重名（如 entity）
    names: tuple = tuple(dict.fromkeys((*CORE_ROLES, *extra)))
    return ParticleTypeSet(names=names)


def handler(args: argparse.Namespace) -> int:
    guard = guard_nl_input(args.nl)
    if not guard.ok:
        print(f"[护栏拒绝] {guard.reason}", file=sys.stderr)   # ⚠ 字段名是 reason（单数）
        return 2

    mode, enabled, disabled, mwarn = resolve(args.mode)
    # 极简模式隐藏门户导航：强制 portal=False，仅产出裸页面
    portal_on = bool(mode.orchestration.get("portal", True))
    if mwarn:
        for w in mwarn:
            print(f"[模式警告] {w}", file=sys.stderr)

    try:
        entity_map = _parse_entity_map(args.entity_map)
        particle_types = _particle_set(args.particle_types, entity_map)
    except ValueError as exc:
        print(f"[参数非法] {exc}", file=sys.stderr)
        return 2

    parsed = parse_nl_to_schema(args.nl, entity_map=entity_map, title=args.title)
    print(f"schema_type   : {parsed.schema.get('type')}")
    print(f"confidence    : {parsed.confidence}")
    print(f"需澄清        : {parsed.needs_clarification}")
    if parsed.notes:
        print(f"notes         : {'; '.join(parsed.notes)}")
    print(f"工作模式      : {mode.id}（portal导航={'开' if portal_on else '关'}; "
          f"启用插件：{', '.join(enabled) or '无'}）")

    result = render_page(parsed.schema, particle_types=particle_types)
    if not result.ok:
        print("[渲染拒绝] 非法 Schema，字段级错误：", file=sys.stderr)
        for err in result.errors:
            print(f"  - {err}", file=sys.stderr)
        return 2

    if args.out:
        path = Path(args.out)
        if path.parent != Path(""):
            path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(result.html, encoding="utf-8")
        print(f"已写出        : {path.resolve()}  ({len(result.html)} 字节)")
    else:
        print(result.html)
```

```
    return 0


def _onboarding_handler(args: argparse.Namespace) -> int:
    """生成引导式导航页（选方向→选工作流→选工作模式→进入工作台）。"""
    result = render_onboarding()
    if not result.ok:
        print("[引导式导航生成失败]", file=sys.stderr)
        for err in result.errors:
            print(f"  - {err}", file=sys.stderr)
        return 2
    if args.out:
        path = Path(args.out)
        if path.parent != Path(""):
            path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(result.html, encoding="utf-8")
        print(f"已写出引导式导航：{path.resolve()}  ({len(result.html)} 字节)")
    else:
        print(result.html)
    return 0


def register(sub: Any) -> None:
    p = sub.add_parser("portal", help="免代码门户")
    sub2 = p.add_subparsers(dest="portal_cmd", required=True)
    r = sub2.add_parser("render", help="NL → HTML")
    r.add_argument("--nl", required=True, help="页面自然语言描述")
    r.add_argument("--out", default=None, help="输出 HTML 路径；缺省打印到 stdout")
    r.add_argument("--title", default=None, help="页面标题")
    r.add_argument("--particle-types", default=None,
                   help=f"追加粒子类型（逗号分隔）；默认仅核心角色 {','.join(CORE_ROLES)}")
    r.add_argument("--entity-map", default=None,
                   help="NL 实体词→粒子类型，如 客户=entity（消费方注入，内核不内置）")
    r.add_argument("--mode", default=None,
                   help="工作模式：standard/ptc/minimal/creative（minimal 关门户导航）")
    r.set_defaults(handler=handler)
    o = sub2.add_parser("onboarding", help="生成引导式导航页（选工作模式）")
    o.add_argument("--out", default=None, help="输出 HTML 路径；缺省打印到 stdout")
    o.set_defaults(handler=_onboarding_handler)
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/commands/selftest.py -----
"""chiyu selftest —— 调 pytest 跑指定包测试（退出码透传）。"""
from __future__ import annotations

import argparse
import subprocess
import sys
from pathlib import Path
from typing import Any, List

# 本文件位于 <仓库根>/packages/chiyu-cli/src/chiyu_cli/commands/selftest.py
```

```
# parents: [0]=commands [1]=chiyu_cli [2]=src [3]=chiyu-cli [4]=packages [5]=仓库根
_ROOT = Path(__file__).resolve().parents[5]

_PACKAGES: dict = {
    "kernel": ("packages/chiyu-kernel/tests",),
    "decision": ("packages/chiyu-decision",),
    "portal": ("packages/chiyu-portal",),
    "cli": ("packages/chiyu-cli",),
}


def _targets(name: str) -> List[str]:
    if name == "all":
        return ["packages"]
    return [str(_ROOT / p) for p in _PACKAGES[name]]


def handler(args: argparse.Namespace) -> int:
    cmd = [
        sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider",
        *_targets(args.package),
    ]
    if args.keyword:
        cmd += ["-k", args.keyword]
    print("$ " + " ".join(cmd))
    completed = subprocess.run(cmd, cwd=str(_ROOT))
    return int(completed.returncode)


def register(sub: Any) -> None:
    p = sub.add_parser("selftest", help="跑包测试")
    p.add_argument("--package", default="all",
                   choices=("all", "kernel", "decision", "portal", "cli"), help="目标包")
    p.add_argument("-k", dest="keyword", default=None, help="pytest -k 过滤")
    p.set_defaults(handler=handler)
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/main.py -----
"""chiyu 命令派发入口。

⛔ 本模块只做派发与参数装配；业务一律下沉到 `commands/*`。
"""
from __future__ import annotations

import argparse
import importlib.util
import sys
from typing import Any, List, Optional

from . import __version__, shell
from .commands import decide, evolution, portal, selftest, modes
```

```
def build_parser() -> argparse.ArgumentParser:
    """构造顶层解析器并注册全部子命令。"""
    parser = argparse.ArgumentParser(
        prog="chiyu",
        description="ChiYu AI 原生决策平台 —— 终端运行入口",
    )
    parser.add_argument("--version", action="version", version=f"chiyu {__version__}")
    sub = parser.add_subparsers(dest="command", required=True)

    decide.register(sub)
    portal.register(sub)
    evolution.register(sub)
    selftest.register(sub)
    modes.register(sub)
    shell.register(sub)

    # `dev`（完整 Python REPL）仅 full 版存在；external 构建不打包该模块。
    # ⛔ 这是构建形态探测（external 版无 dev.pyc），不是运行期动态派发。
    if importlib.util.find_spec("chiyu_cli.commands.dev") is not None:
        from .commands import dev

        dev.register(sub)

    return parser


def main(argv: Optional[List[str]] = None) -> int:
    """CLI 主入口。返回进程退出码。"""
    parser = build_parser()
    args: Any = parser.parse_args(argv)   # required=True ⇒ 必有 handler
    return int(args.handler(args) or 0)


if __name__ == "__main__":  # pragma: no cover
    sys.exit(main())
# ----- FILE: packages/chiyu-cli/src/chiyu_cli/shell.py -----
"""受限命令壳 —— ⛔ 白名单外一律拒绝（技术上封死 raw import 逃逸）。

external（外部顾问版）只提供本壳，不提供完整 Python REPL；
因此外部顾问**无法**借 `chiyu shell` 执行 `import chiyu_kernel` 读取源码。
"""
from __future__ import annotations

import argparse
import shlex
import sys
from typing import Any, List

from .commands.decide import handler as _decide
```

```
from .commands.evolution import handler as _readiness
from .commands.portal import handler as _render
from .commands.selftest import handler as _selftest
from .commands._plugins import resolve

HELP = """可用命令：
  decide "议题"                        跑决策脊柱
  render "页面描述" [--out p.html]     NL → HTML
  readiness [--assert]                开闸前置认证
  selftest [kernel|decision|portal|cli|all]
  help                                显示本帮助
  exit                                退出
⛔ 仅以上命令可用；不支持任意 Python 语句与 import。
"""

_EXIT = -1          # 哨兵：退出循环


def _run(argv: List[str]) -> int:
    """把壳内一行解析为受控调用。未知命令 ⇒ 拒绝并退 2。"""
    if not argv:
        return 0
    head, rest = argv[0], argv[1:]

    if head == "decide":
        topic = rest[0] if rest else ""
        if not topic:
            print('用法：decide "议题"', file=sys.stderr)
            return 2
        return int(_decide(argparse.Namespace(
            topic=topic, scores=None, threshold=0.79, decision_id=None, json=False)))

    if head == "render":
        nl = rest[0] if rest else ""
        if not nl:
            print('用法：render "页面描述" [--out p.html]', file=sys.stderr)
            return 2
        out = None
        if "--out" in rest:
            idx = rest.index("--out")
            out = rest[idx + 1] if idx + 1 < len(rest) else None
        return int(_render(argparse.Namespace(
            nl=nl, out=out, title=None, particle_types=None)))

    if head == "readiness":
        return int(_readiness(argparse.Namespace(
            assert_="--assert" in rest, json=False)))

    if head == "selftest":
        pkg = rest[0] if rest else "all"
```

```
        return int(_selftest(argparse.Namespace(package=pkg, keyword=None)))

    if head == "help":
        print(HELP)
        return 0

    if head == "exit":
        return _EXIT

    print(f"[拒绝] 不支持的命令 {head!r}；输入 help 查看可用命令。", file=sys.stderr)
    return 2


def run_shell(mode_id: str | None = None) -> int:
    """启动受限命令壳。返回 0（正常退出）。"""
    mode, enabled, disabled, mwarn = resolve(mode_id)
    if mwarn:
        for w in mwarn:
            print(f"[模式警告] {w}", file=sys.stderr)
    banner = (f"chiyu shell（受限命令壳）— 工作模式：{mode.id}（{mode.label}）；"
              f"启用插件：{', '.join(enabled) or '无'}。输入 help 查看命令，exit 退出。")
    if mode.orchestration.get("portal_hidden"):
        banner += "  极简模式：门户导航已隐藏。"
    print(banner)
    while True:
        try:
            line = input("chiyu> ")
        except (EOFError, KeyboardInterrupt):
            print()
            return 0
        try:
            argv = shlex.split(line)
        except ValueError as exc:
            print(f"[拒绝] 无法解析：{exc}", file=sys.stderr)
            continue
        rc = _run(argv)
        if rc == _EXIT:
            return 0
    return 0


def handler(args: Any) -> int:
    return run_shell(getattr(args, "mode", None))


def register(sub: Any) -> None:
    p = sub.add_parser("shell", help="受限命令壳（两版共有）")
    p.add_argument("--mode", default=None,
                   help="工作模式：standard/ptc/minimal/creative（缺省=standard）")
    p.set_defaults(handler=handler)
```